Import the necessary libraries

In [1]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as st
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
%matplotlib inline
sns.set(style="whitegrid")
```

Import Dataset

In [2]:
```
heart_data = pd.read_csv(r'D:\fsds\projects\datasets\heart.csv')
```

In [3]:
```
heart_data.head()
```

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2

Exploratory Data Analysis

In [4]:
```
heart_data.shape
```

Out[4]:
```
(1025, 14)
```

In [5]:
```
heart_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       1025 non-null   int64  
 1   sex       1025 non-null   int64  
 2   cp        1025 non-null   int64  
 3   trestbps  1025 non-null   int64  
 4   chol      1025 non-null   int64  
 5   fbs       1025 non-null   int64  
 6   restecg   1025 non-null   int64  
 7   thalach   1025 non-null   int64  
 8   exang     1025 non-null   int64  
 9   oldpeak   1025 non-null   float64
 10  slope     1025 non-null   int64  
 11  ca        1025 non-null   int64  
 12  thal      1025 non-null   int64  
 13  target    1025 non-null   int64  
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

In [6]:
```
heart_data.dtypes
```

Out[6]:
```
age           int64
sex           int64
cp            int64
trestbps      int64
chol          int64
fbs           int64
restecg       int64
thalach       int64
exang         int64
oldpeak     float64
slope         int64
ca            int64
thal          int64
target        int64
dtype: object
```

In [7]:
```
heart_data.describe()
```

Out[7]:

	age	sex	cp	trestbps	chol	fbs	10
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.00000	1025.000000	
mean	54.434146	0.695610	0.942439	131.611707	246.00000	0.149268	
std	9.072290	0.460373	1.029641	17.516718	51.59251	0.356527	
min	29.000000	0.000000	0.000000	94.000000	126.00000	0.000000	
25%	48.000000	0.000000	0.000000	120.000000	211.00000	0.000000	
50%	56.000000	1.000000	1.000000	130.000000	240.00000	0.000000	
75%	61.000000	1.000000	2.000000	140.000000	275.00000	0.000000	
max	77.000000	1.000000	3.000000	200.000000	564.00000	1.000000	

In [8]:
```
heart_data.columns
```

Out[8]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

Univariant Analysis

In [9]:
```
heart_data['target'].nunique()
```

Out[9]:
```
2
```

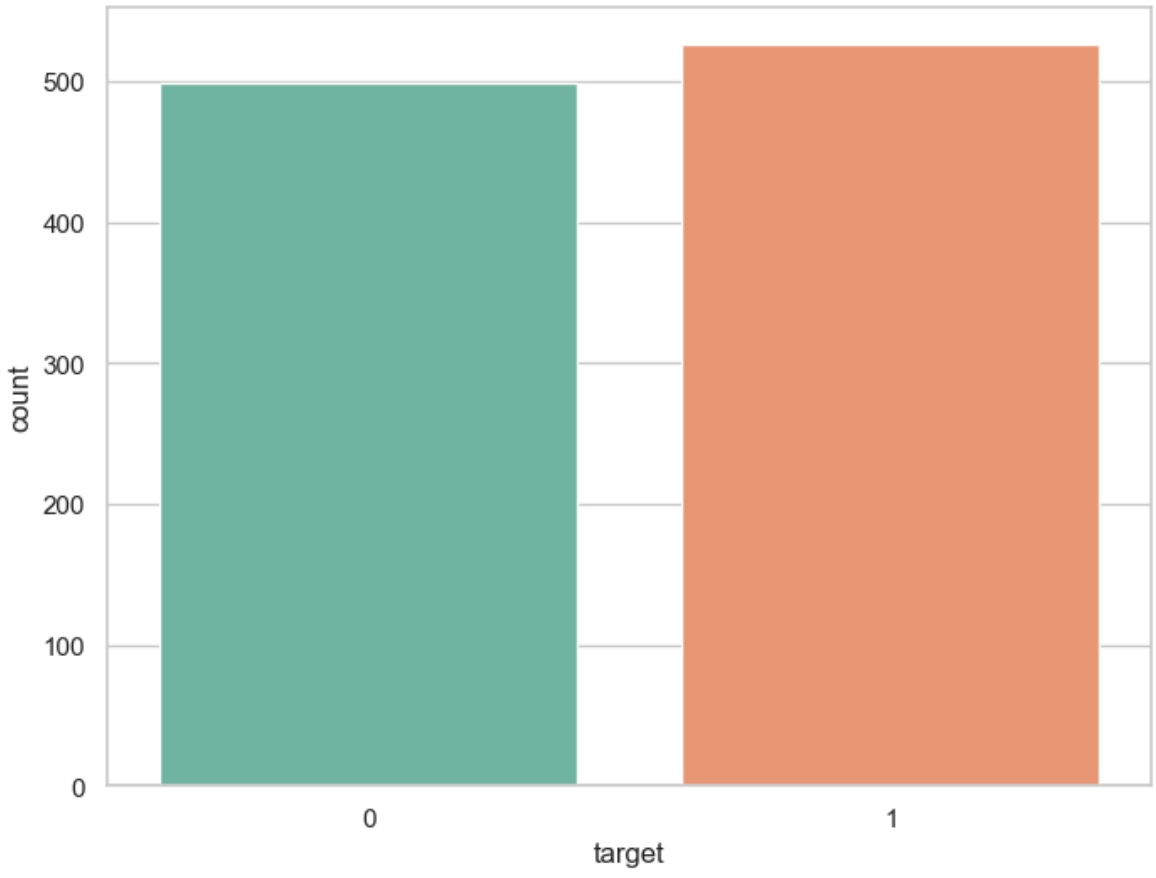
In [10]:
```
heart_data['target'].unique()
```

Out[10]:
```
array([0, 1], dtype=int64)
```

In [11]:
```
heart_data['target'].value_counts()
```

Out[11]:
```
target
1    526
0    499
Name: count, dtype: int64
```

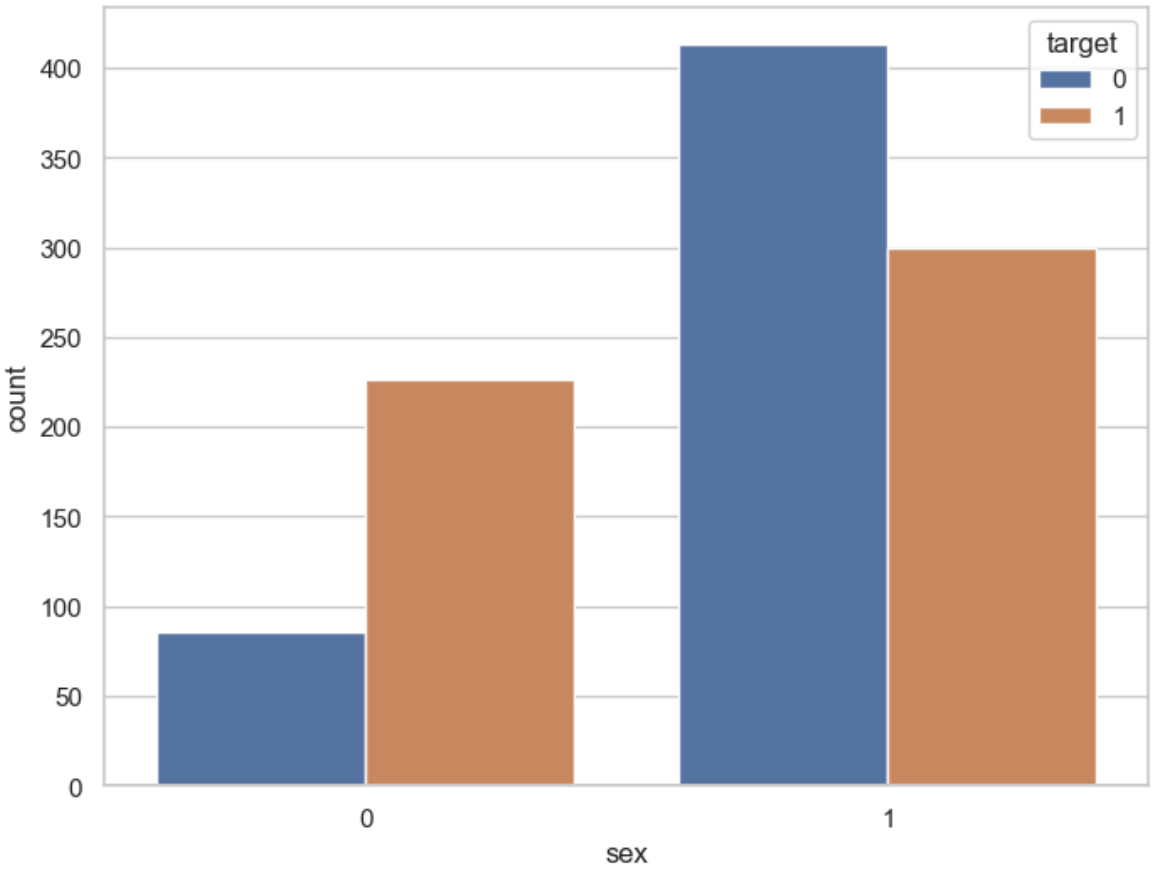
In [12]:
```
f, ax=plt.subplots(figsize=(8,6))
sx=sns.countplot(x='target', data=heart_data, palette='Set2')
plt.show()
```



In [13]:
```
heart_data.groupby('sex')['target'].value_counts()
```

Out[13]:
```
sex  target
0    1         226
     0          86
1    0         413
     1         300
Name: count, dtype: int64
```

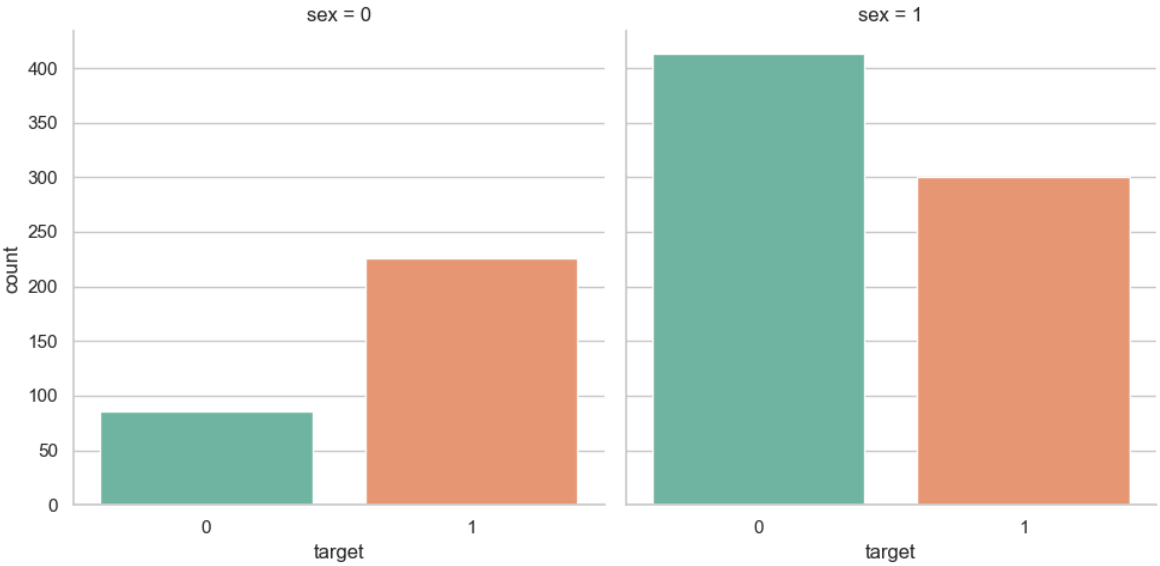
In [14]:
```
f, ax=plt.subplots(figsize=(8,6))
sx=sns.countplot(x='sex', data=heart_data, hue='target')
plt.show()
```



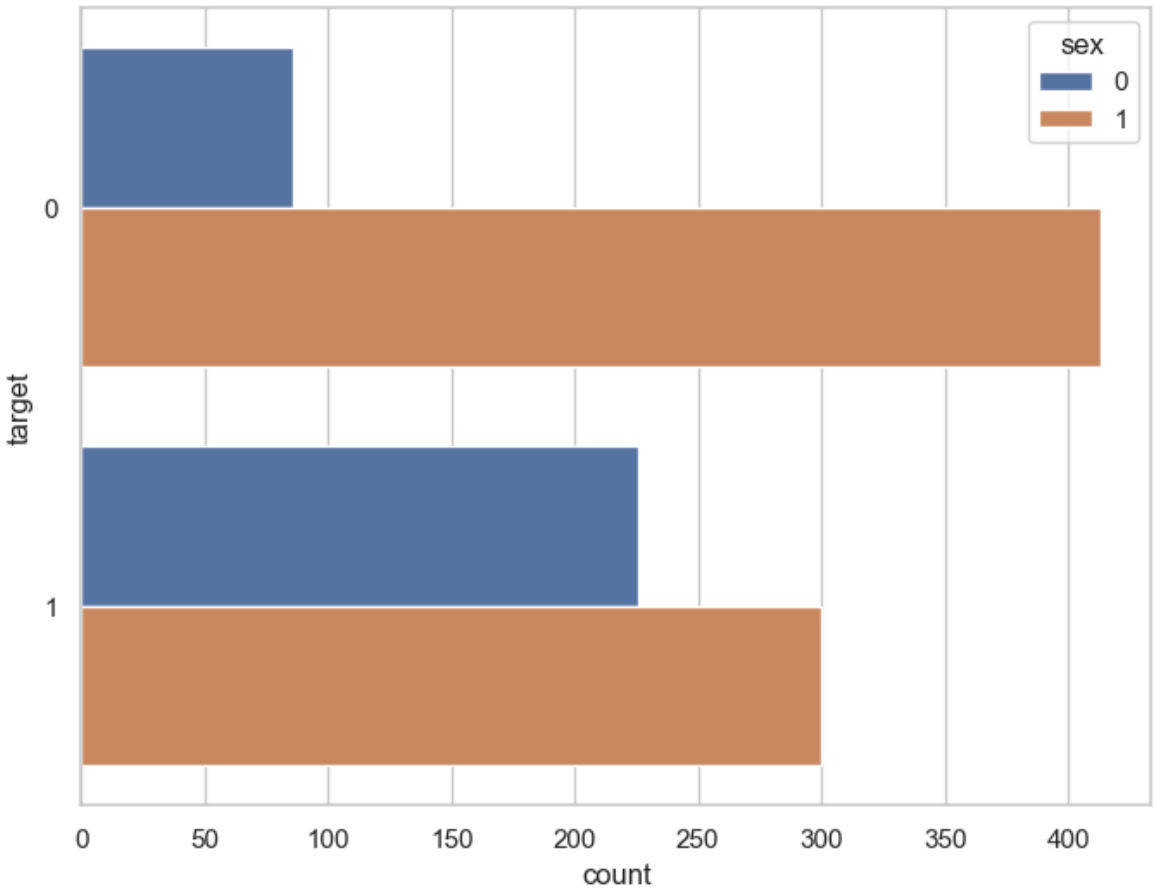
Insigths

- The above plot confirms that:
  - Out of 313 females, 88 have heart disease and 225 do not.
  - Out of 713 males, 300 have heart disease and 413 do not.

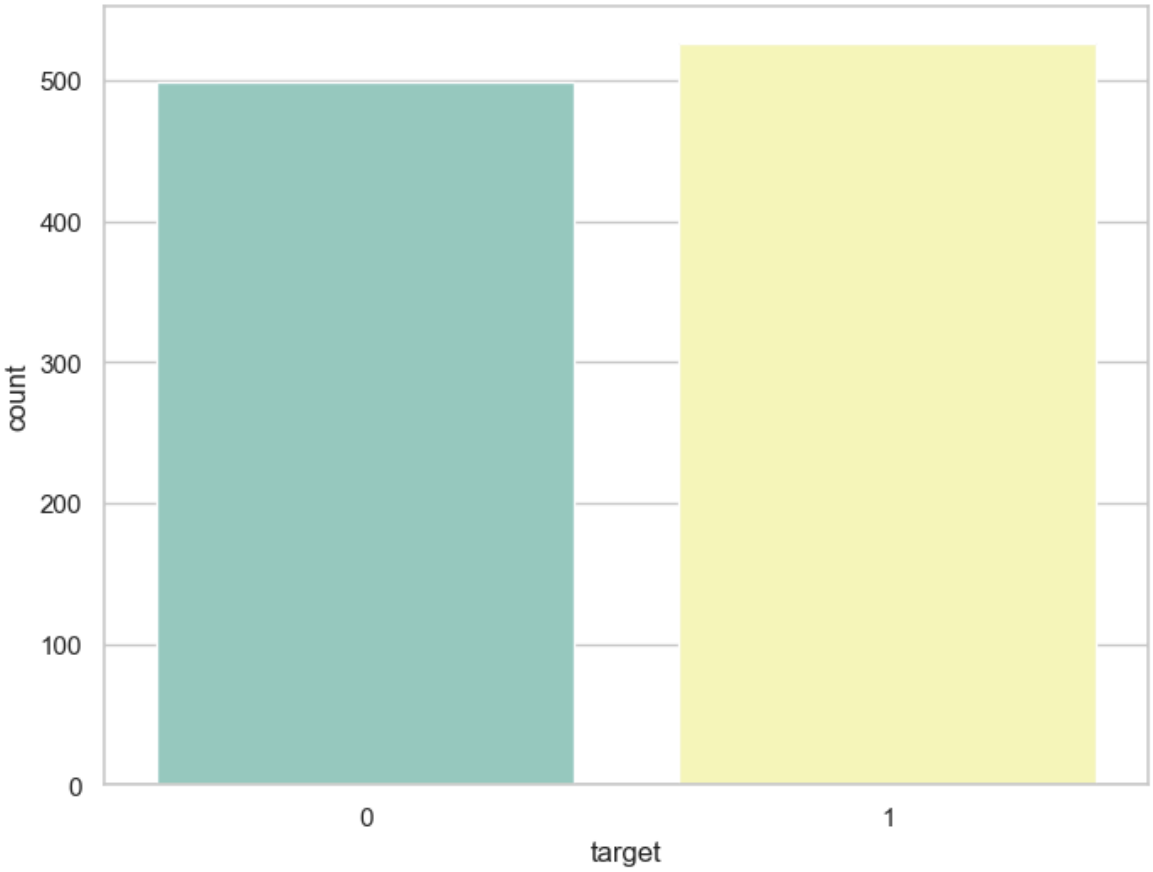
In [15]:
```
ax=sns.catplot(x='target',data=heart_data, col='sex', kind='count', height=5, as
```



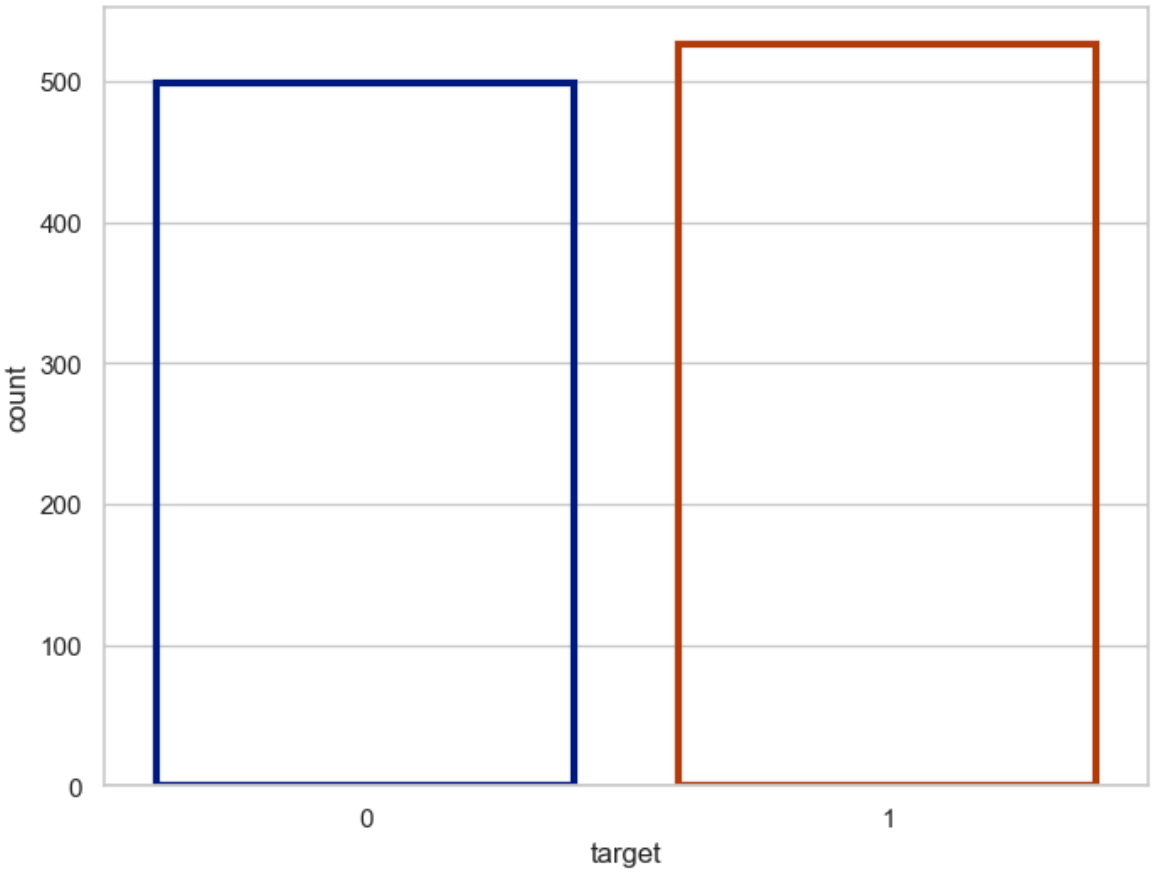
In [16]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(y='target', hue='sex', data=heart_data)
plt.show()
```



In [17]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', palette='Set3', data=heart_data)
plt.show()
```

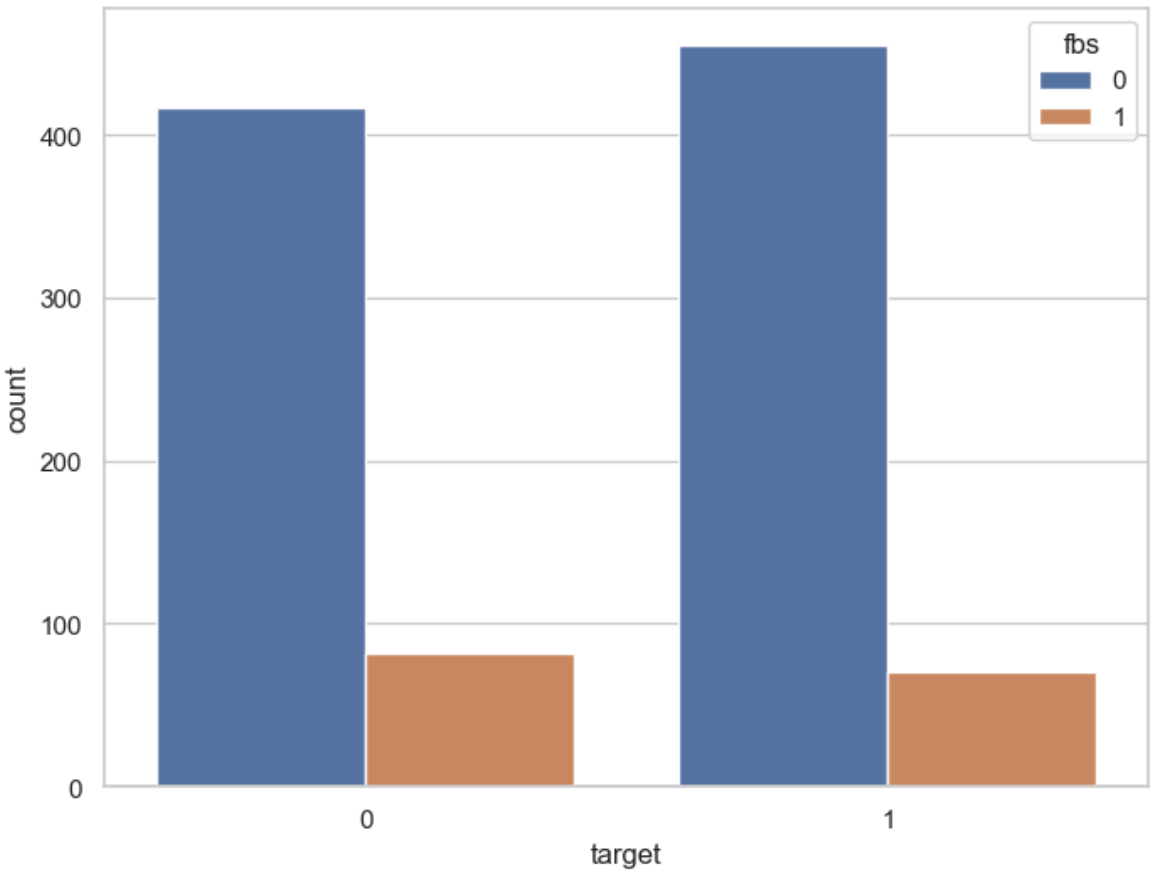


In [18]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', data=heart_data, facecolor=(0,0,0,0), linewidth=3
plt.show()
```

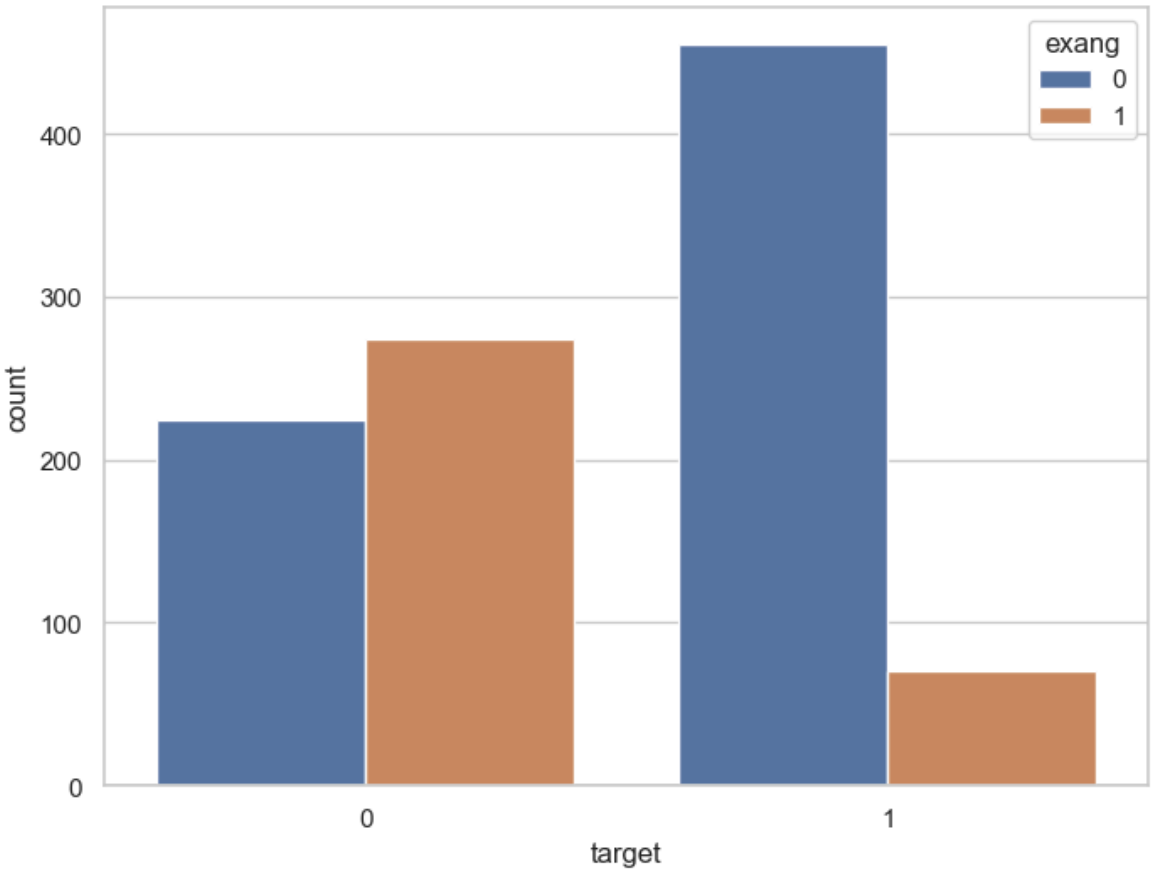


In [19]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', hue='fbs', data=heart_data)
```

```
plt.show()
```



In [20]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.countplot(x='target', hue='exang', data=heart_data)
plt.show()
```



In [21]:
```
heart_data['target'].value_counts()
```

Out[21]:
```
target
1    526
0    499
Name: count, dtype: int64
```

Bivariant Analysis

In [22]:
```
correlation = heart_data.corr() #corr() is used to find the correlation between
```

In [23]:
```
correlation['target'].sort_values(ascending=False)
```

Out[23]:
```
target      1.000000
cp          0.434854
thalach     0.422895
slope       0.345512
restecg     0.134468
fbs        -0.041164
chol       -0.099966
trestbps   -0.138772
age        -0.229324
sex        -0.279501
thal       -0.337838
ca         -0.382085
exang      -0.438029
oldpeak    -0.438441
Name: target, dtype: float64
```

In [24]:
```
df=heart_data
```

In [25]:
```
df['cp'].nunique
```

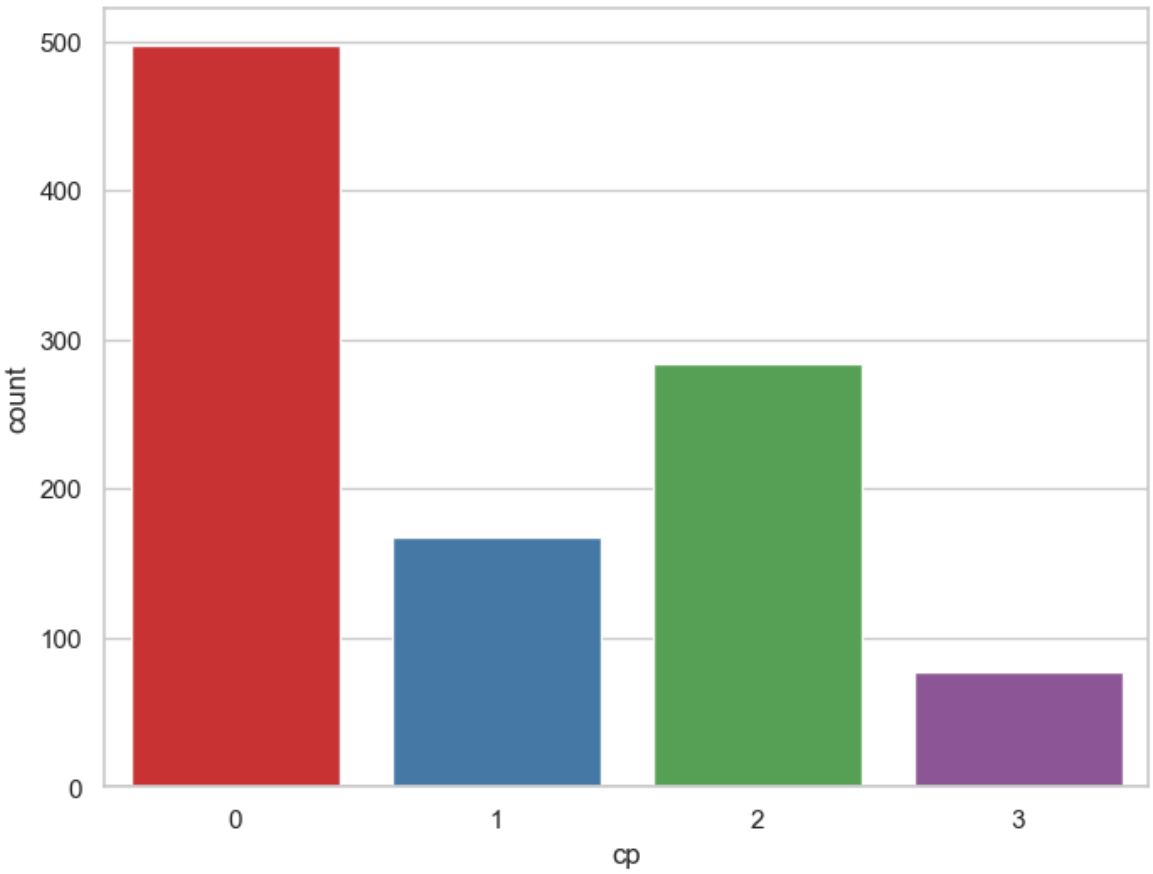
Out[25]:
```
<bound method IndexOpsMixin.nunique of 0       0
1       0
2       0
3       0
4       0
       ..
1020    1
1021    0
1022    0
1023    0
1024    0
Name: cp, Length: 1025, dtype: int64>
```

In [26]:
```
df['cp'].value_counts()
```

Out[26]:
```
cp
0    497
2    284
1    167
3     77
Name: count, dtype: int64
```

In [27]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.countplot(x='cp', data=df, palette='Set1')
```

```
plt.show()
```



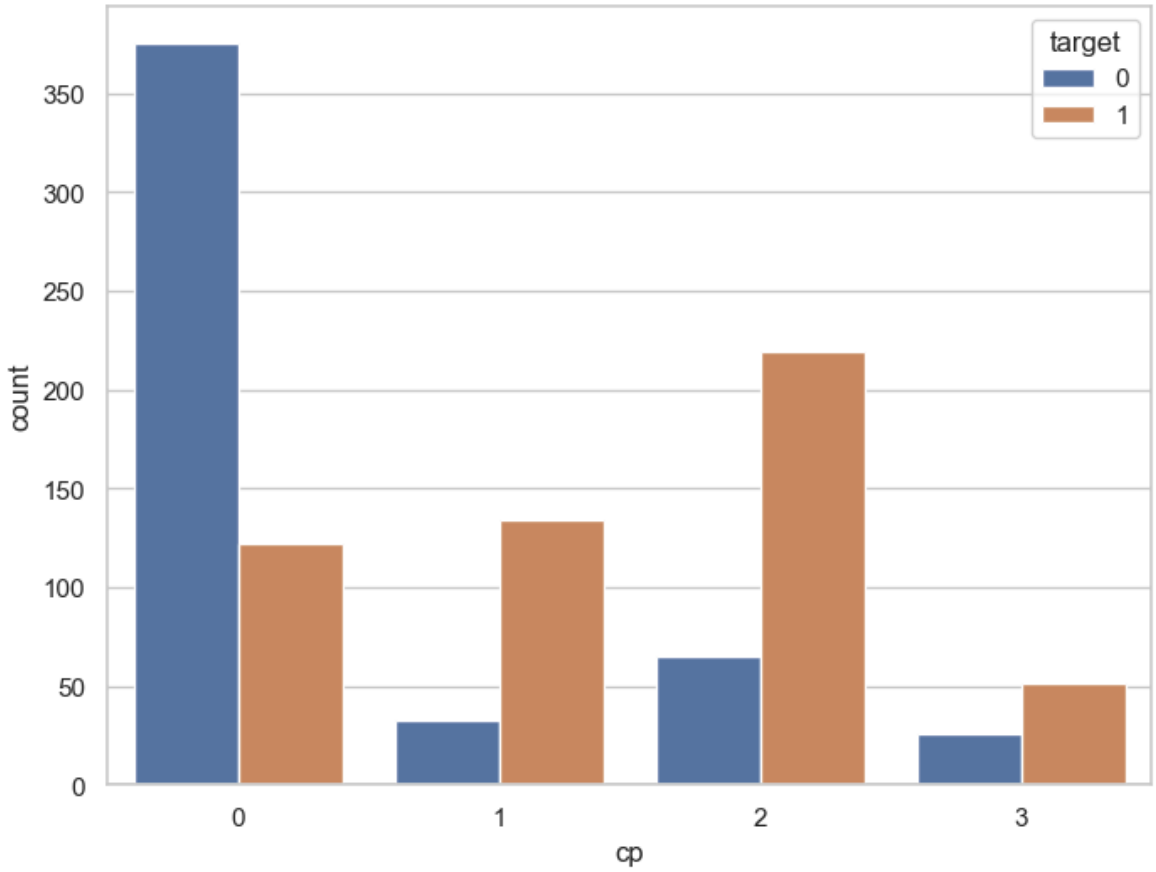
In [28]:
```
df.groupby('cp')['target'].value_counts()
```

Out[28]:
```
cp  target
0   0         375
    1         122
1   1         134
    0          33
2   1         219
    0          65
3   1          51
    0          26
Name: count, dtype: int64
```

Comments

- `cp` contains 4 integer values: 0,1,2 and 3
- `target` contains 2 integer values: 0 and 1
- the above analysis give us an idea of the distribution of the `target` variable grouped by the `cp` variable

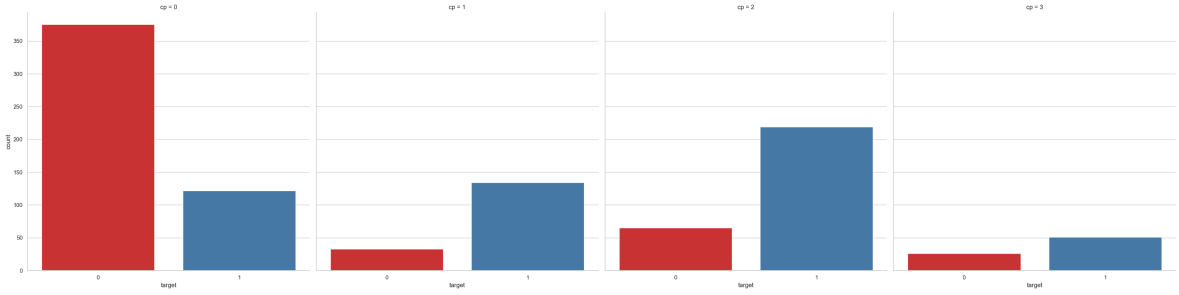
In [29]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="cp", hue="target", data=df)
plt.show()
```



Interpretation

- we can conclude that `target` variables are plotted wrt `cp`

In [30]:
```
ax = sns.catplot(x="target", col="cp", data=df, kind="count", height=8, aspect=1
```



Analysis of `target` and `thalach`

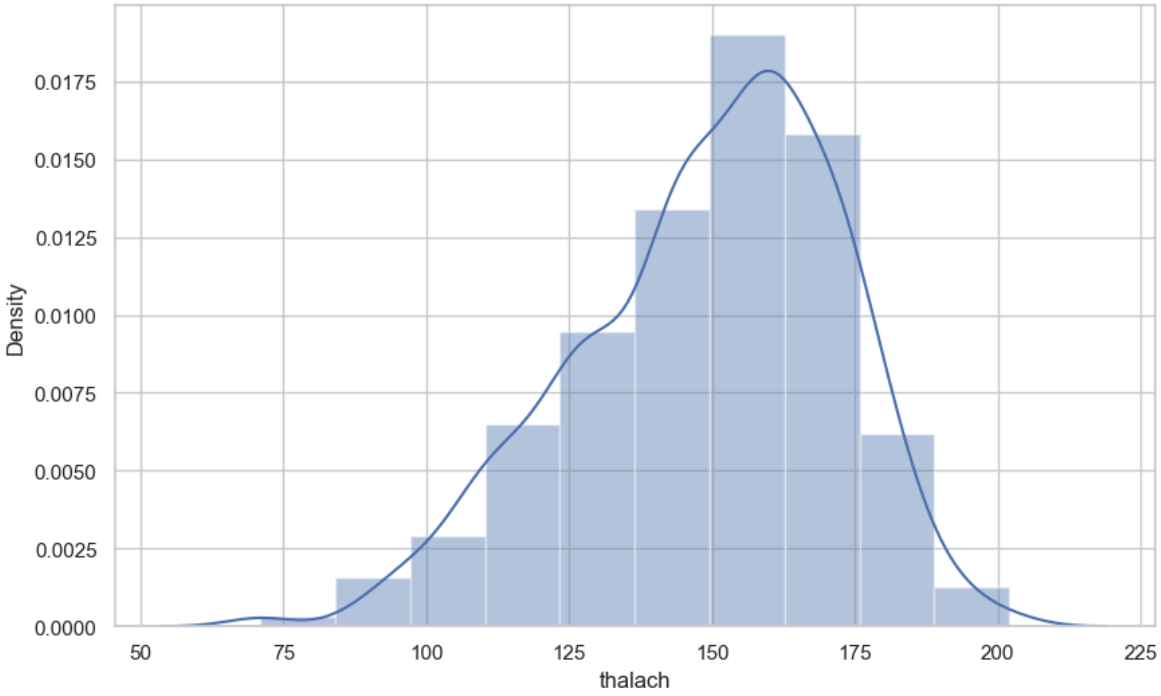
- `thalach` is the maximum heart rate achieved during exercise.
- `target` is the presence of heart disease.
- we can chcek the no of unique values in `thalach` as shown below:

In [31]:
```
df['thalach'].nunique()
```

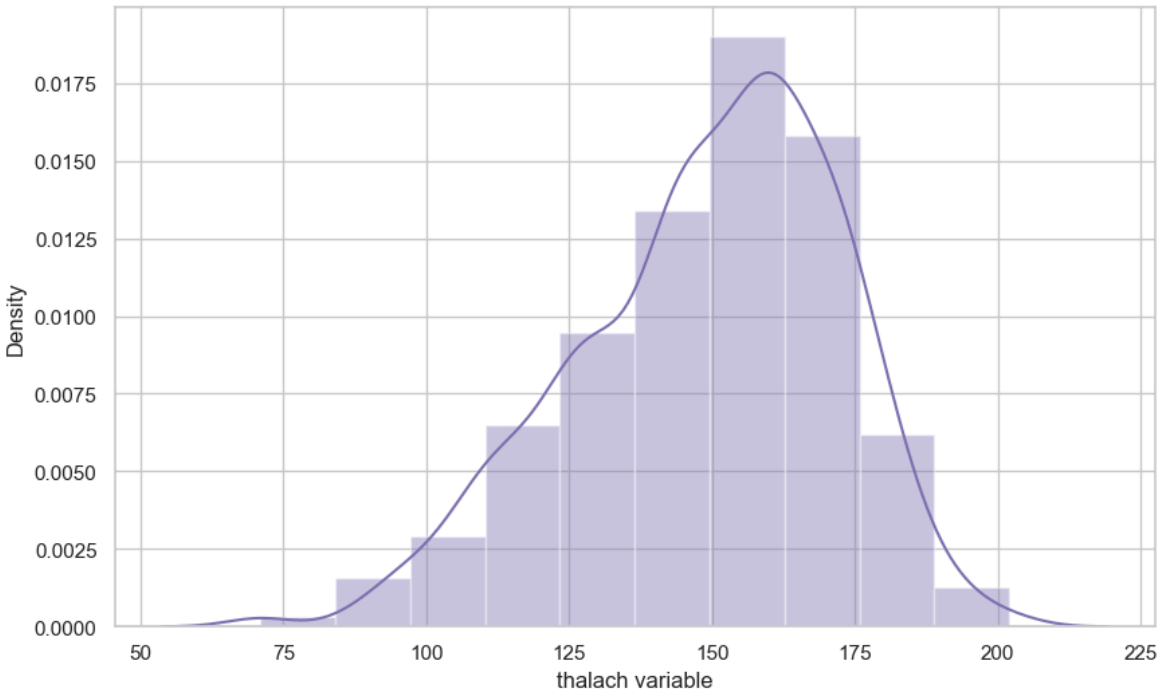
Out[31]: 91

In [32]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
```

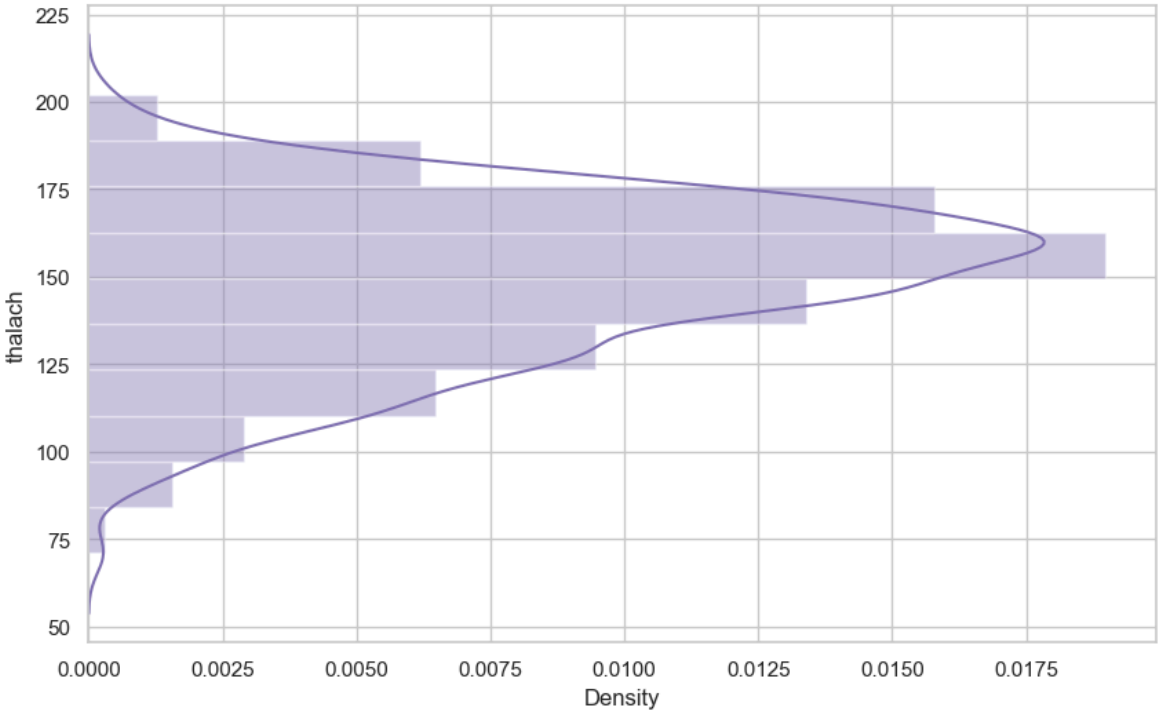
```
ax = sns.distplot(x, bins=10)
plt.show()
```



In [33]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.distplot(x, bins=10, color='m')
plt.show()
```



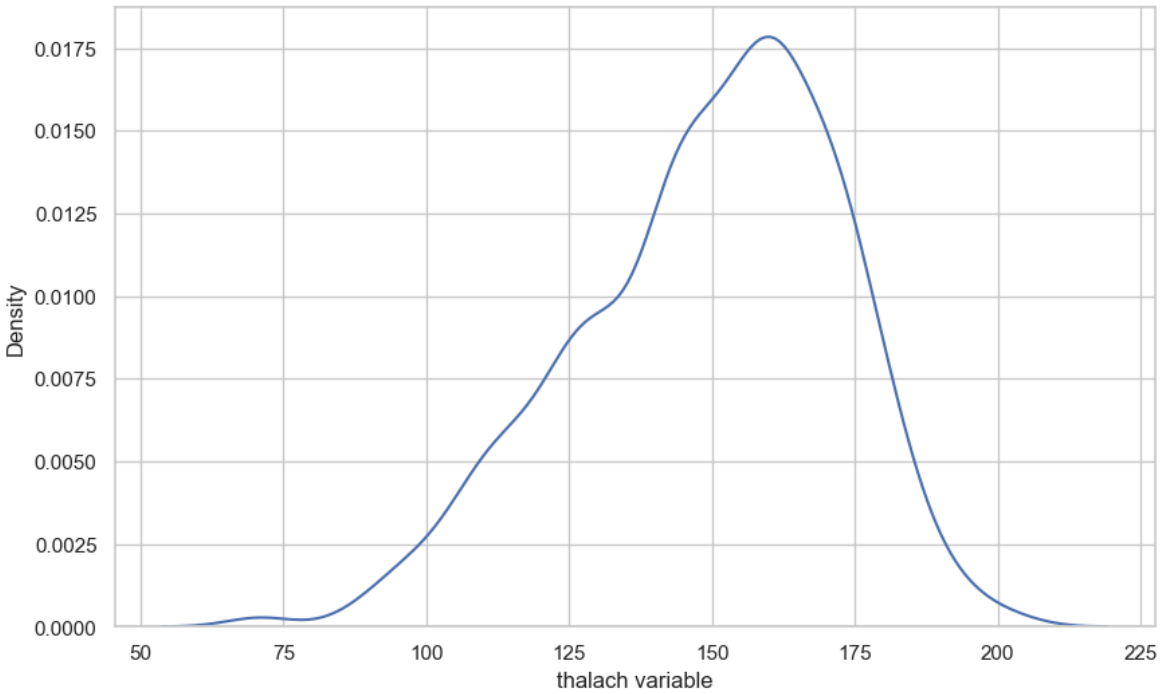
In [34]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, bins=10, vertical=True, color='m')
plt.show()
```



Seaborn Kernel Density Estimation (KDE) Plot

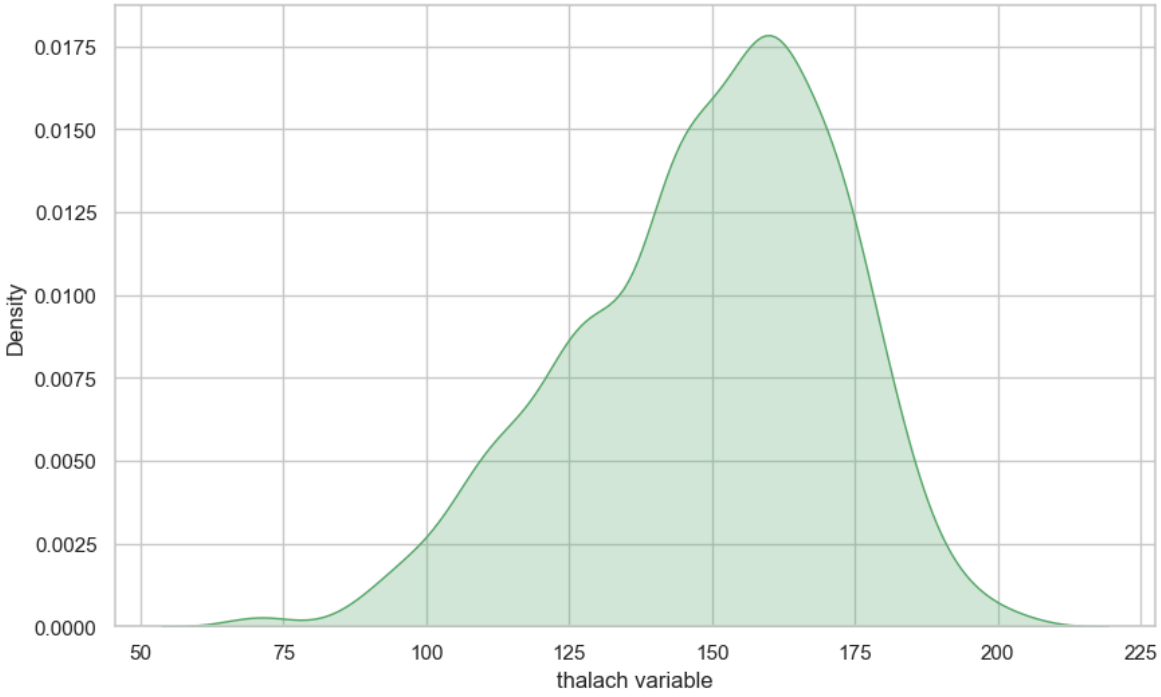
- The kernel density estimate (KDE) plot is a useful tool for plotting the shape of a distribution.
- The KDE plot plots the density of observations on one axis with height along the other axis.

In [35]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x)
plt.show()
```



We can shade under the density curve and use a different color as follows:

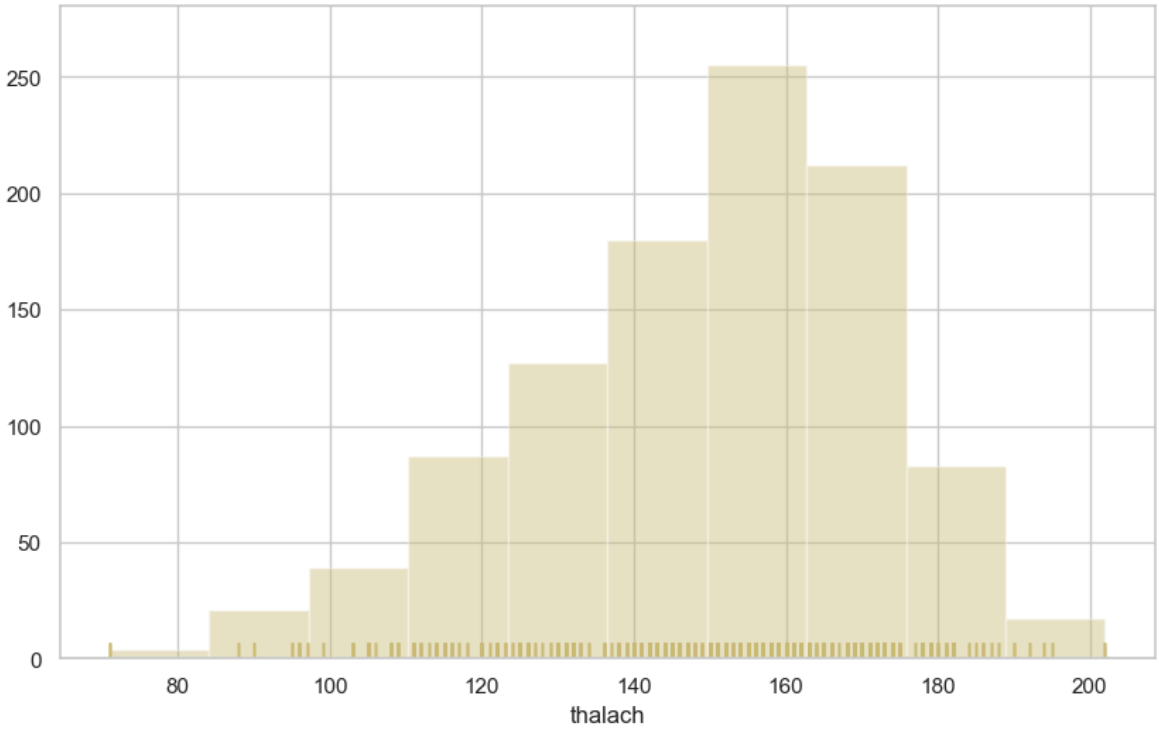
In [36]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x, shade=True, color='g')
plt.show()
```



Histogram

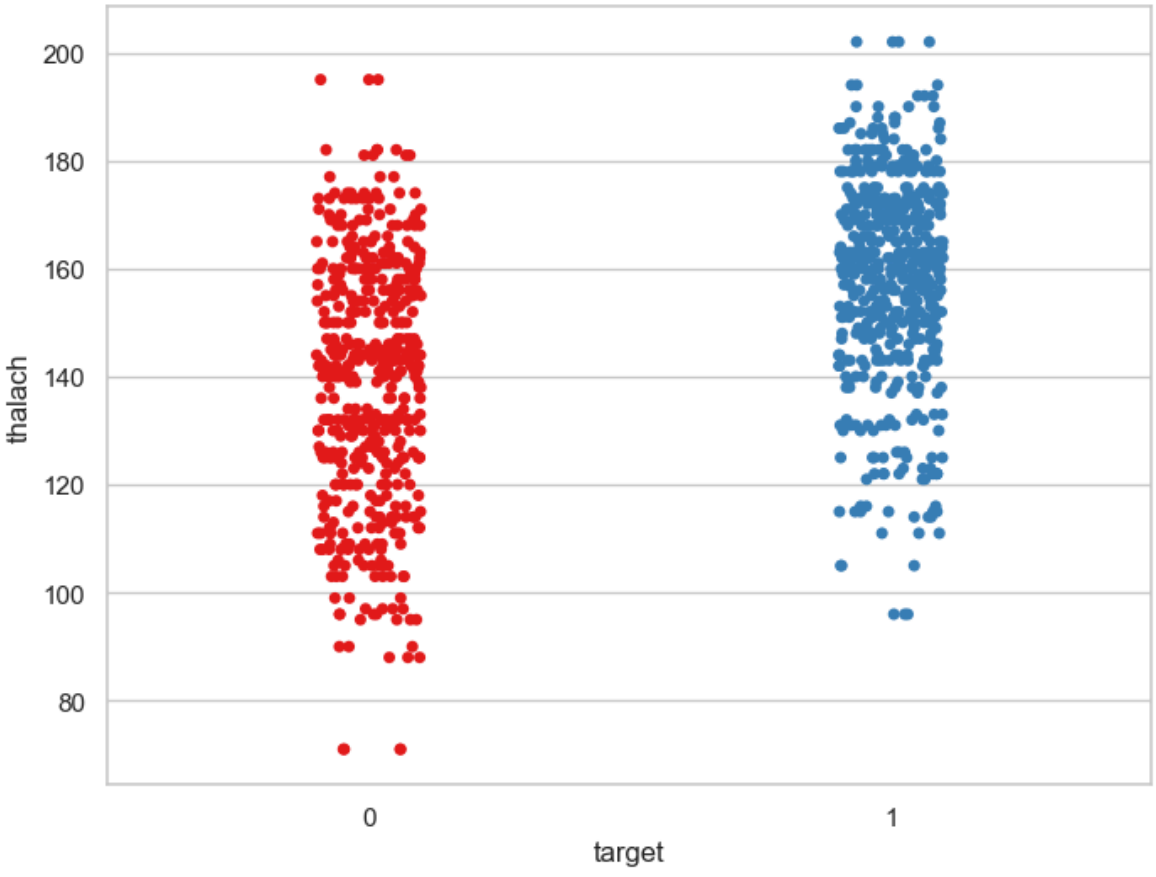
- A histogram represents the distribution of data by forming bins along the range of the data and then drawing bars to show the number of observations that fall in each bin.
- We can plot a histogram as follows :

In [37]:
```
f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, kde=False, rug=True, bins=10, color='y')
plt.show()
```



Visualize frequency distribution of `thalach` variable wrt `target`

In [38]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, palette="Set1")
plt.show()
```

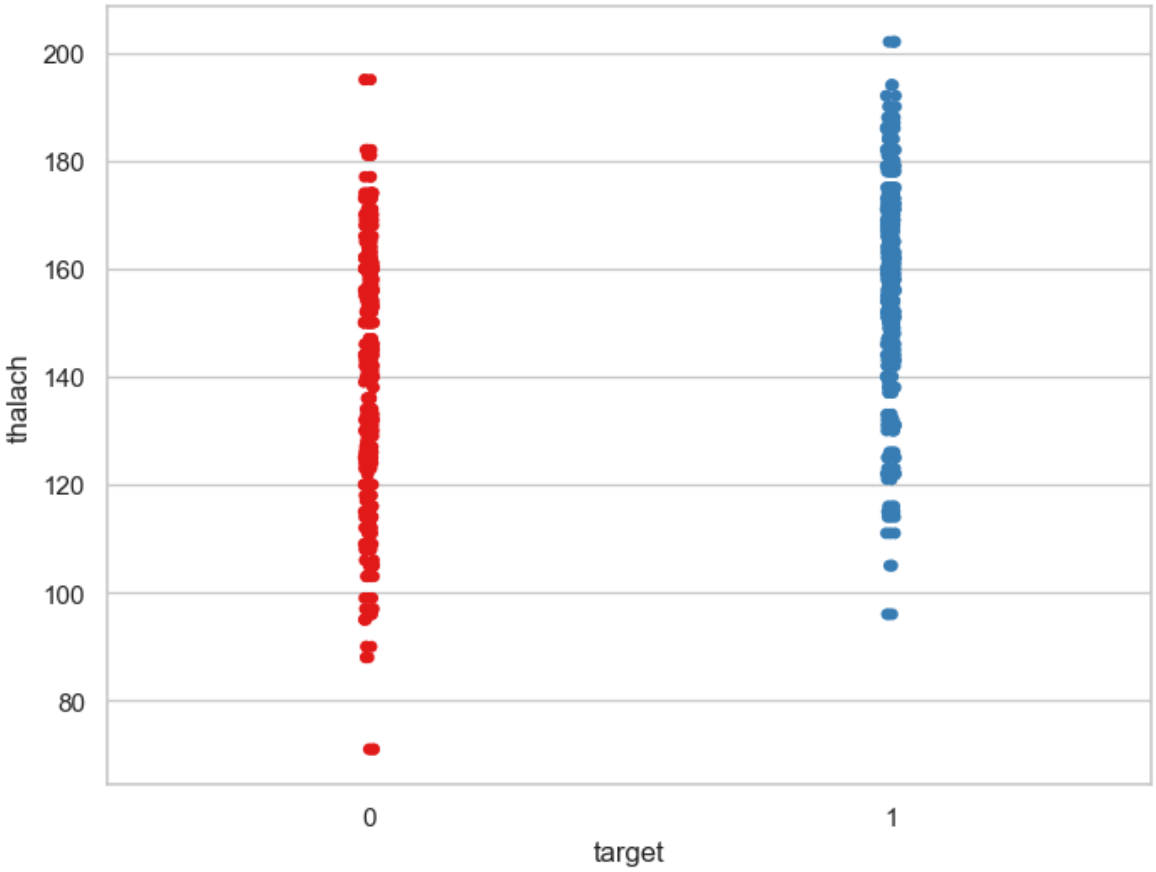


Interpretation

- We can see that those people suffering from heart disease (target = 1) have relatively higher heart rate (thalach) as compared to people who are not suffering from heart disease (target = 0).

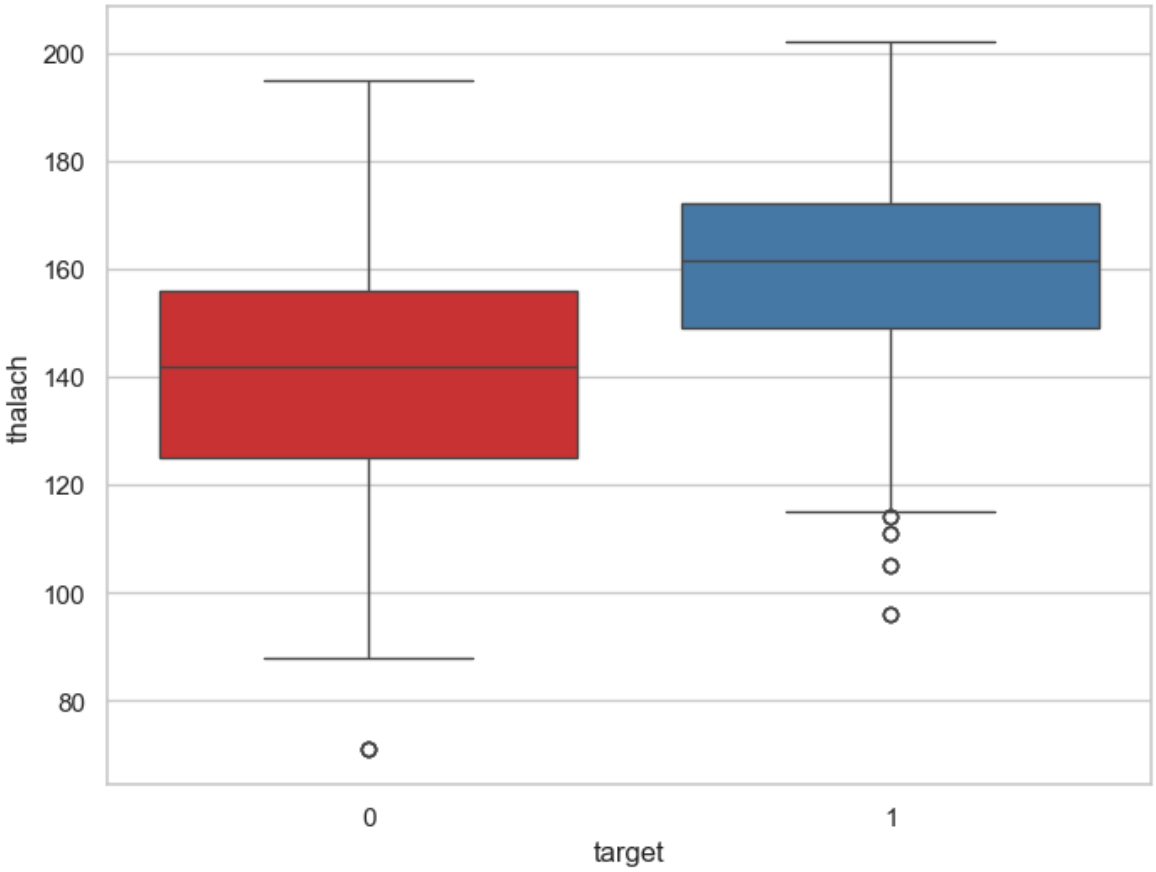
We can add jitter to bring out the distribution of values as follows :

In [39]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, jitter = 0.01, palette='Set1')
plt.show()
```



Visualize distribution of `thalach` variable wrt `target` with boxplot

In [40]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="target", y="thalach", data=df, palette="Set1")
plt.show()
```



Interpretation

The above boxplot confirms our finding that people suffering from heart disease (target = 1) have relatively higher heart rate (thalach) as compared to people who are not suffering from heart disease (target = 0).

Findings of Bivariate Analysis

Findings of Bivariate Analysis are as follows –

- There is no variable which has strong positive correlation with `target` variable.
- There is no variable which has strong negative correlation with `target` variable.
- There is no correlation between `target` and `fbs`.
- The `cp` and `thalach` variables are mildly positively correlated with `target` variable.
- We can see that the `thalach` variable is slightly negatively skewed.
- The people suffering from heart disease (target = 1) have relatively higher heart rate (thalach) as compared to people who are not suffering from heart disease (target = 0).
- The people suffering from heart disease (target = 1) have relatively higher heart rate (thalach) as compared to people who are not suffering from heart disease (target = 0).

Multi Variant Analysis

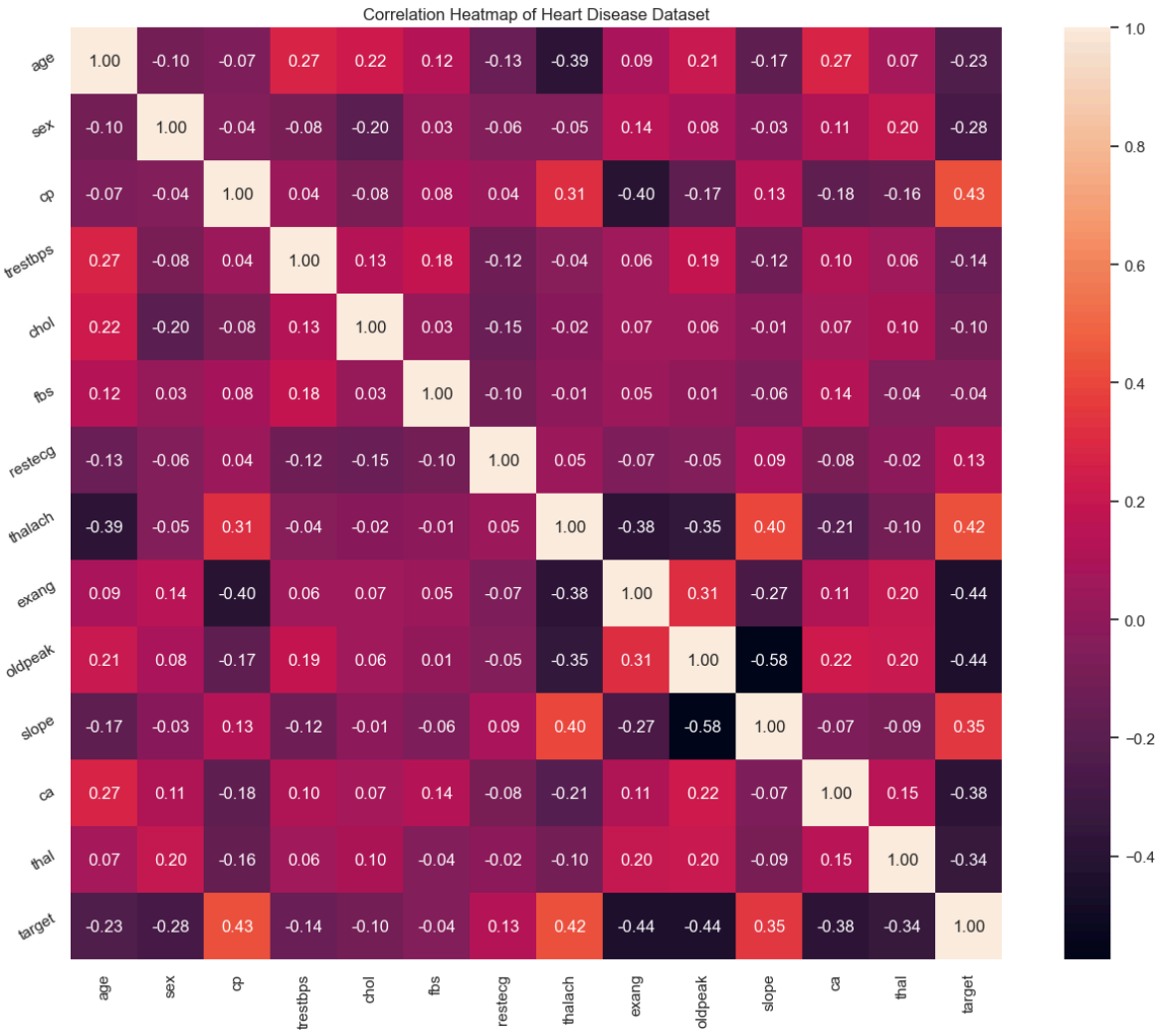
Main objective is to find out relations between variables in the dataset

Discover patterns and relationships

- An important step in EDA is to discover patterns and relationships between variables in the dataset.
- I will use `heat map` and `pair plot` to discover the patterns and relationships in the dataset.
- First of all, I will draw a `heat map`.

Heatmap

In [41]:
```
plt.figure(figsize=(16,12))
plt.title('Correlation Heatmap of Heart Disease Dataset')
a = sns.heatmap(correlation, square=True, annot=True, fmt='.2f', linecolor='whit
a.set_xticklabels(a.get_xticklabels(), rotation=90)
a.set_yticklabels(a.get_yticklabels(), rotation=30)
plt.show()
```


Correlation Heatmap of Heart Disease Dataset
age 1.00 -0.10 -0.07 0.27 0.22 0.12 -0.13 -0.39 0.09 0.21 -0.17 0.27 0.07 -0.23
sex -0.10 1.00 -0.04 -0.08 -0.20 0.03 -0.06 -0.05 0.14 0.08 -0.03 0.11 0.20 -0.28
cp -0.07 -0.04 1.00 0.04 -0.08 0.08 0.04 0.31 -0.40 -0.17 0.13 -0.18 -0.16 0.43
trestbps 0.27 -0.08 0.04 1.00 0.13 0.18 -0.12 -0.04 0.06 0.19 -0.12 0.10 0.06 -0.14
chol 0.22 -0.20 -0.08 0.13 1.00 0.03 -0.15 -0.02 0.07 0.06 -0.01 0.07 0.10 -0.10
fbs 0.12 0.03 0.08 0.18 0.03 1.00 -0.10 -0.01 0.05 0.01 -0.06 0.14 -0.04 -0.04
restecg -0.13 -0.06 0.04 -0.12 -0.15 -0.10 1.00 0.05 -0.07 -0.05 0.09 -0.08 -0.02 0.13
thalach -0.39 -0.05 0.31 -0.04 -0.02 -0.01 0.05 1.00 -0.38 -0.35 0.40 -0.21 -0.10 0.42
exang 0.09 0.14 -0.40 0.06 0.07 0.05 -0.07 -0.38 1.00 0.31 -0.27 0.11 0.20 -0.44
oldpeak 0.21 0.08 -0.17 0.19 0.06 0.01 -0.05 -0.35 0.31 1.00 -0.58 0.22 0.20 -0.44
slope -0.17 -0.03 0.13 -0.12 -0.01 -0.06 0.09 0.40 -0.27 -0.58 1.00 -0.07 -0.09 0.35
ca 0.27 0.11 -0.18 0.10 0.07 0.14 -0.08 -0.21 0.11 0.22 -0.07 1.00 0.15 -0.38
thal 0.07 0.20 -0.16 0.06 0.10 -0.04 -0.02 -0.10 0.20 0.20 -0.09 0.15 1.00 -0.34
target -0.23 -0.28 0.43 -0.14 -0.10 -0.04 0.13 0.42 -0.44 -0.44 0.35 -0.38 -0.34 1.00
age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal target
1.0 0.8 0.6 0.4 0.2 0.0 −0.2 −0.4

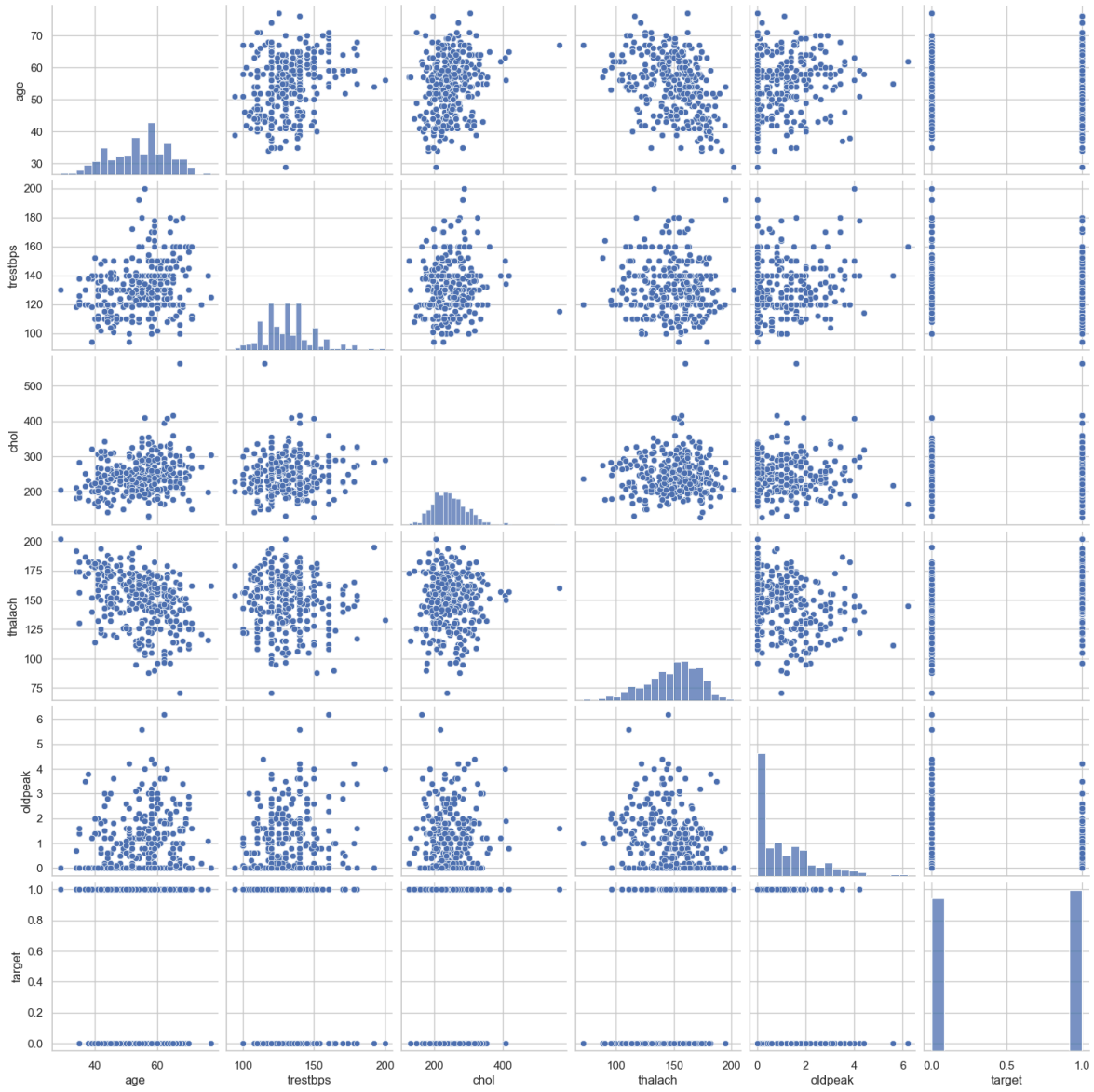
Interpretation

From the above correlation heat map, we can conclude that :-

- `target` and `cp` variable are mildly positively correlated (correlation coefficient = 0.43).
- `target` and `thalach` variable are also mildly positively correlated (correlation coefficient = 0.42).
- `target` and `slope` variable are weakly positively correlated (correlation coefficient = 0.35).
- `target` and `exang` variable are mildly negatively correlated (correlation coefficient = -0.44).
- `target` and `oldpeak` variable are also mildly negatively correlated (correlation coefficient = -0.43).
- `target` and `ca` variable are weakly negatively correlated (correlation coefficient = -0.39).
- `target` and `thal` variable are also waekly negatively correlated (correlation coefficient = -0.34).

Pair Plot

In [42]:
```
num_var = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'target' ]
sns.pairplot(df[num_var], kind='scatter', diag_kind='hist')
plt.show()
```



Comment

- I have defined a variable `num_var`. Here `age`, `trestbps`, `chol`, `thalach` and `oldpeak` are numerical variables and `target` is the categorical variable.
- So, I wll check relationships between these variables.

Analysis of `age` and other variables

In [43]:
```
df['age'].nunique()
```

Out[43]: 41

In [44]:
```
df['age'].describe()
```

Out[44]:
```
count    1025.000000
mean       54.434146
std         9.072290
min        29.000000
25%        48.000000
50%        56.000000
75%        61.000000
max        77.000000
Name: age, dtype: float64
```

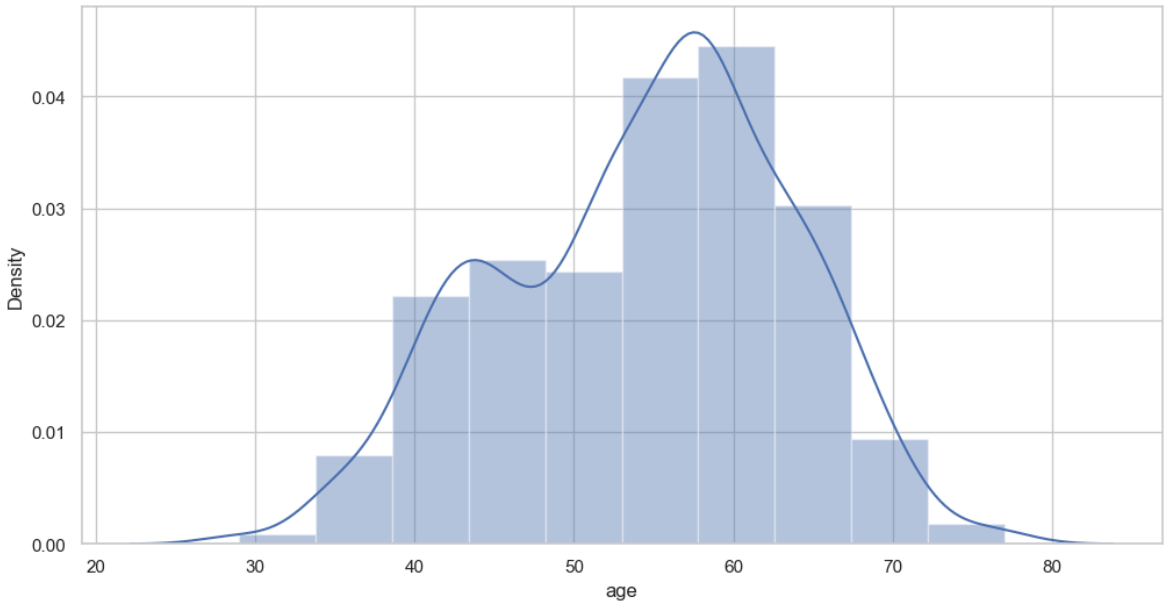
Interpretation

- The mean value of the `age` variable is 54.37 years.
- The minimum and maximum values of `age` are 29 and 77 years.

Plot the distribution of `age` variable

Now, I will plot the distribution of `age` variable to view the statistical properties.

In [45]:
```
f, ax = plt.subplots(figsize=(12,6))
x = df['age']
ax = sns.distplot(x, bins=10)
plt.show()
```



Interpretation

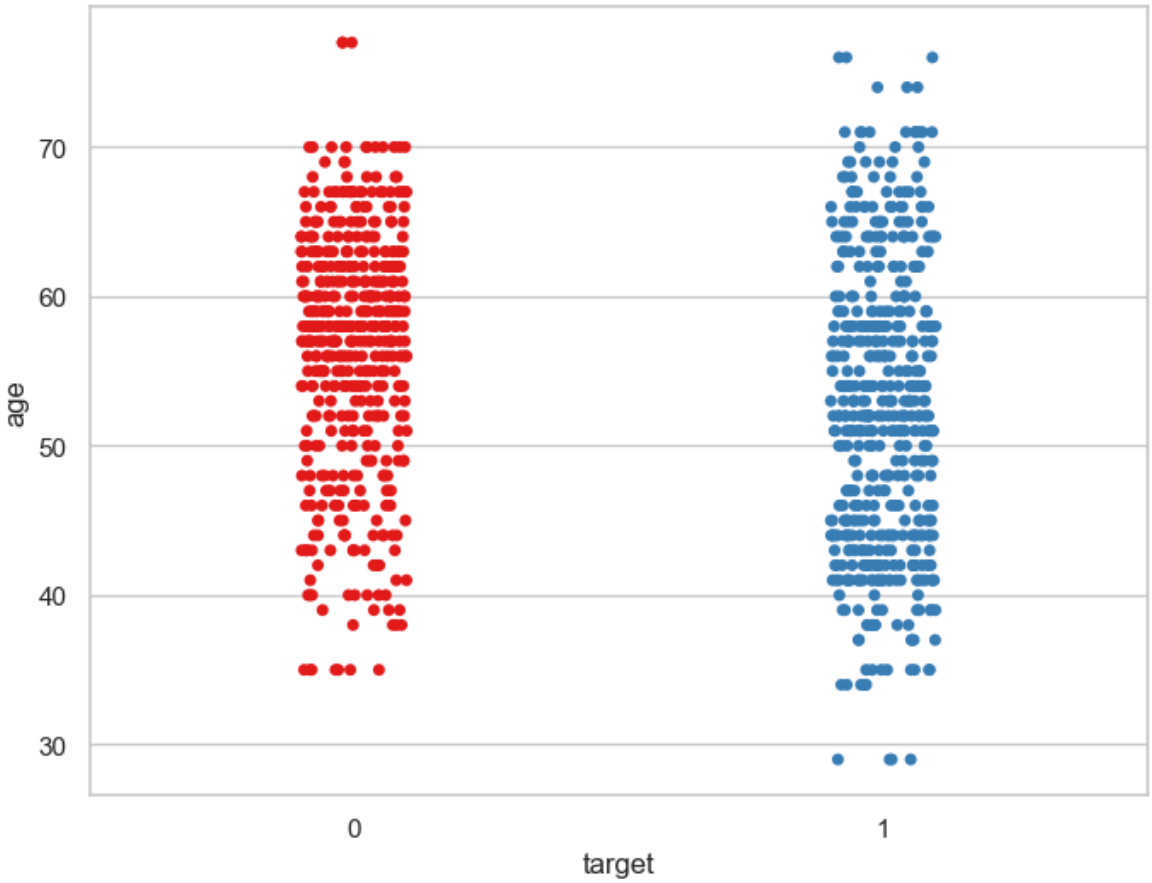
- The `age` variable distribution is approximately normal.

Analyze `age` and `target` variable

Visualize frequency distribution of `age` variable wrt `target`

In [46]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="age", data=df, palette="Set1")
```

```
plt.show()
```

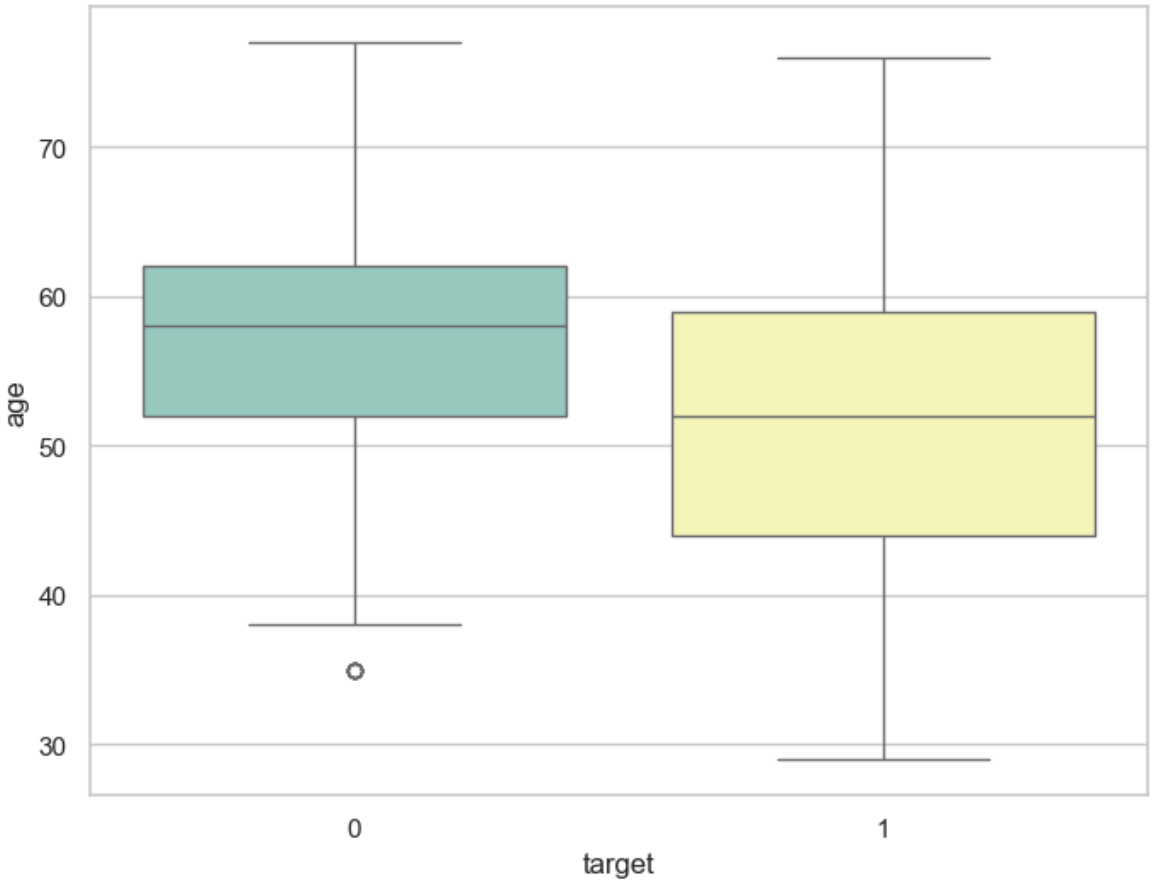


Interpretation

- We can see that the people suffering from heart disease (target = 1) and people who are not suffering from heart disease (target = 0) have comparable ages.

Visualize distribution of `age` variable wrt `target` with boxplot

In [47]:
```
f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="target", y="age", data=df, palette="Set3")
plt.show()
```



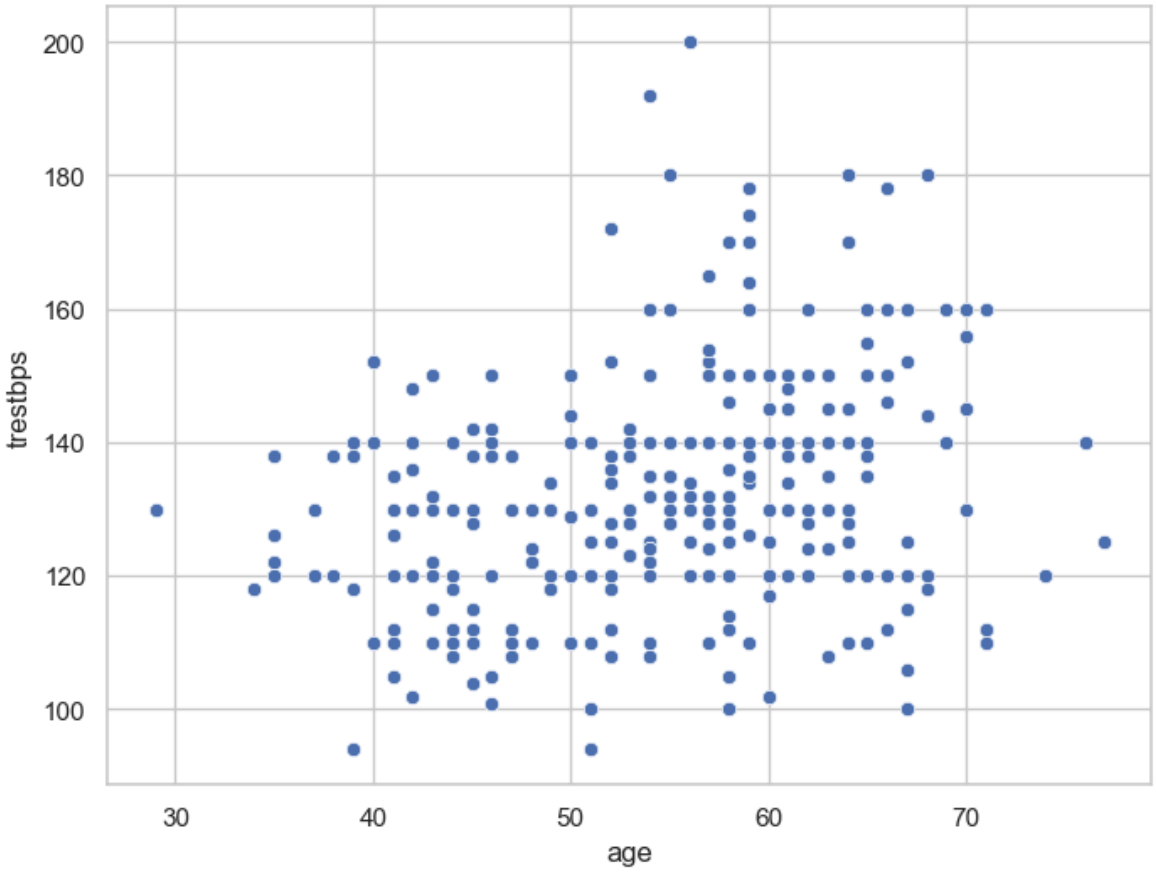
Interpretation

- The above boxplot tells two different things :
  - The mean age of the people who have heart disease is less than the mean age of the people who do not have heart disease.
  - The dispersion or spread of age of the people who have heart disease is greater than the dispersion or spread of age of the people who do not have heart disease.

Analyze `age` and `trestbps` variable

I will plot a scatterplot to visualize the relationship between `age` and `trestbps` variable.

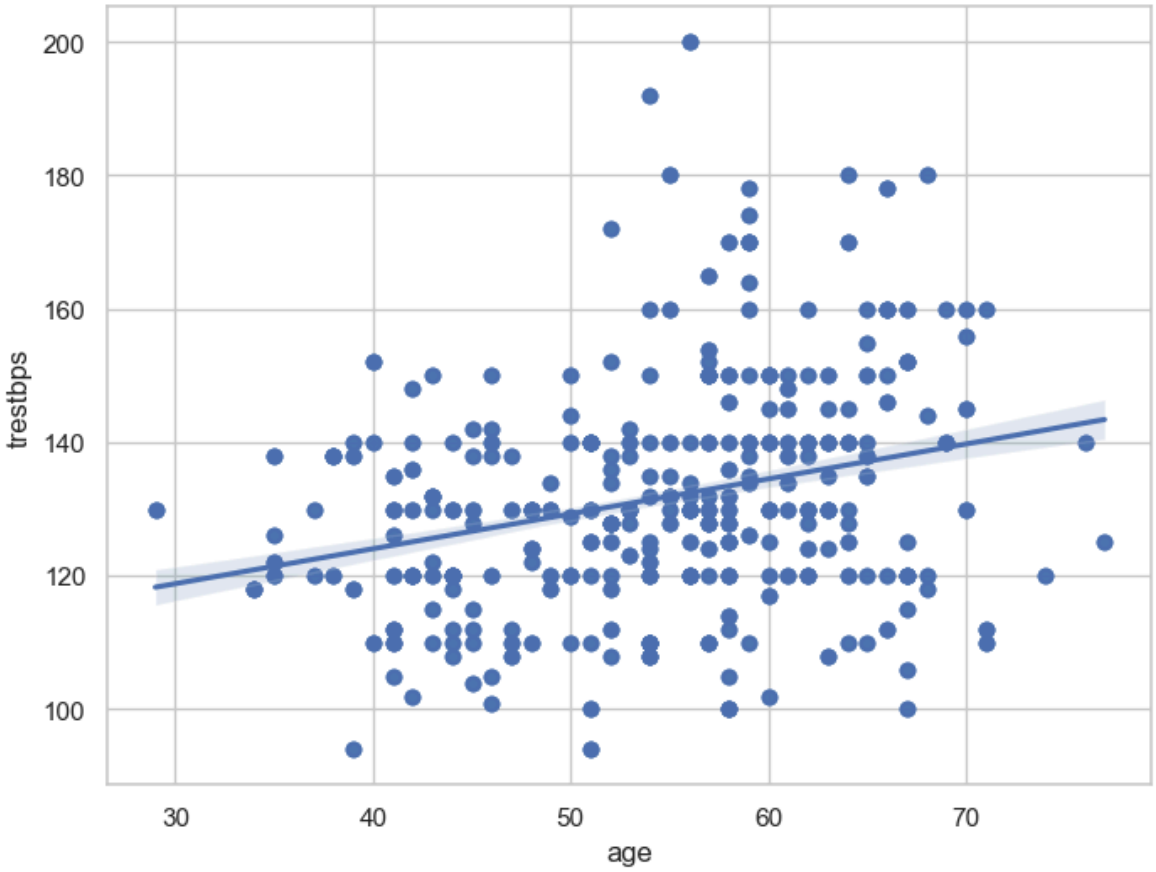
In [48]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="age", y="trestbps", data=df)
plt.show()
```



Interpretation

- The above scatter plot shows that there is no correlation between `age` and `trestbps` variable.

In [49]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="trestbps", data=df)
plt.show()
```

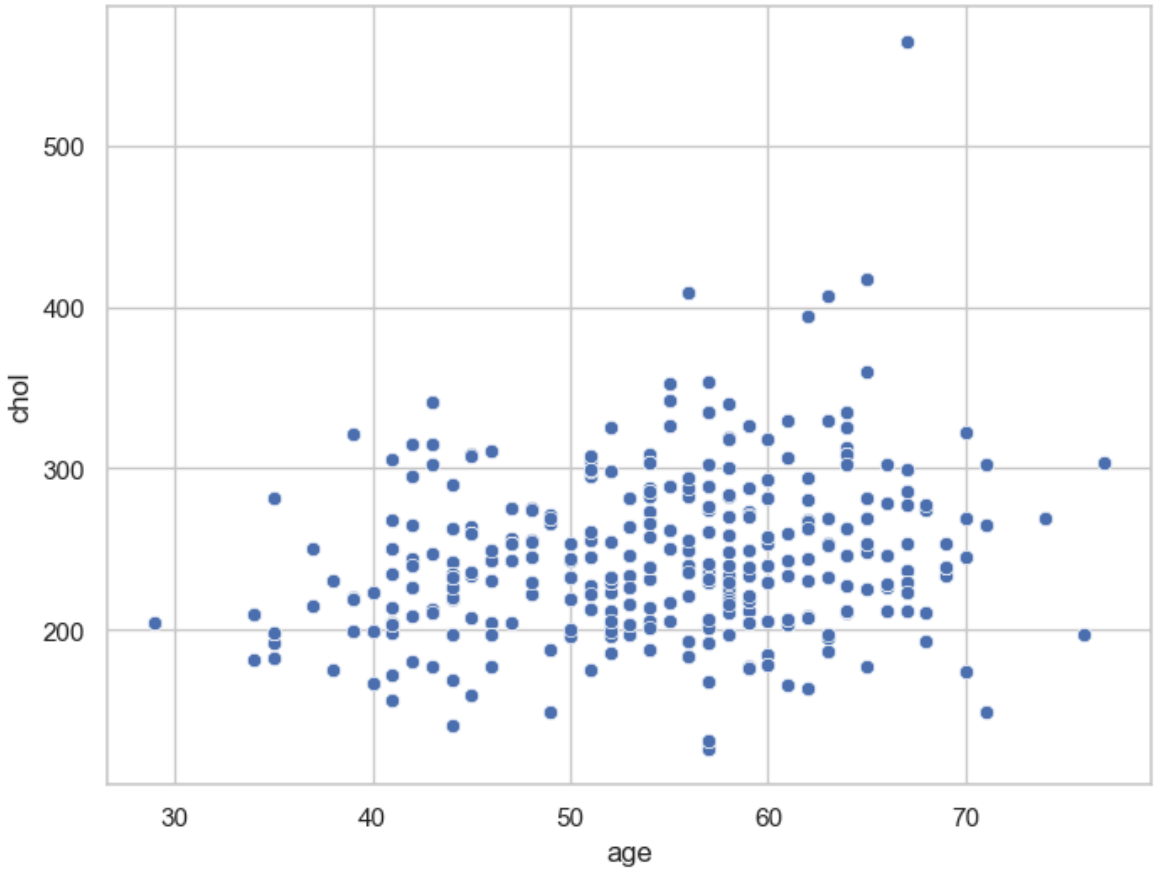


Interpretation

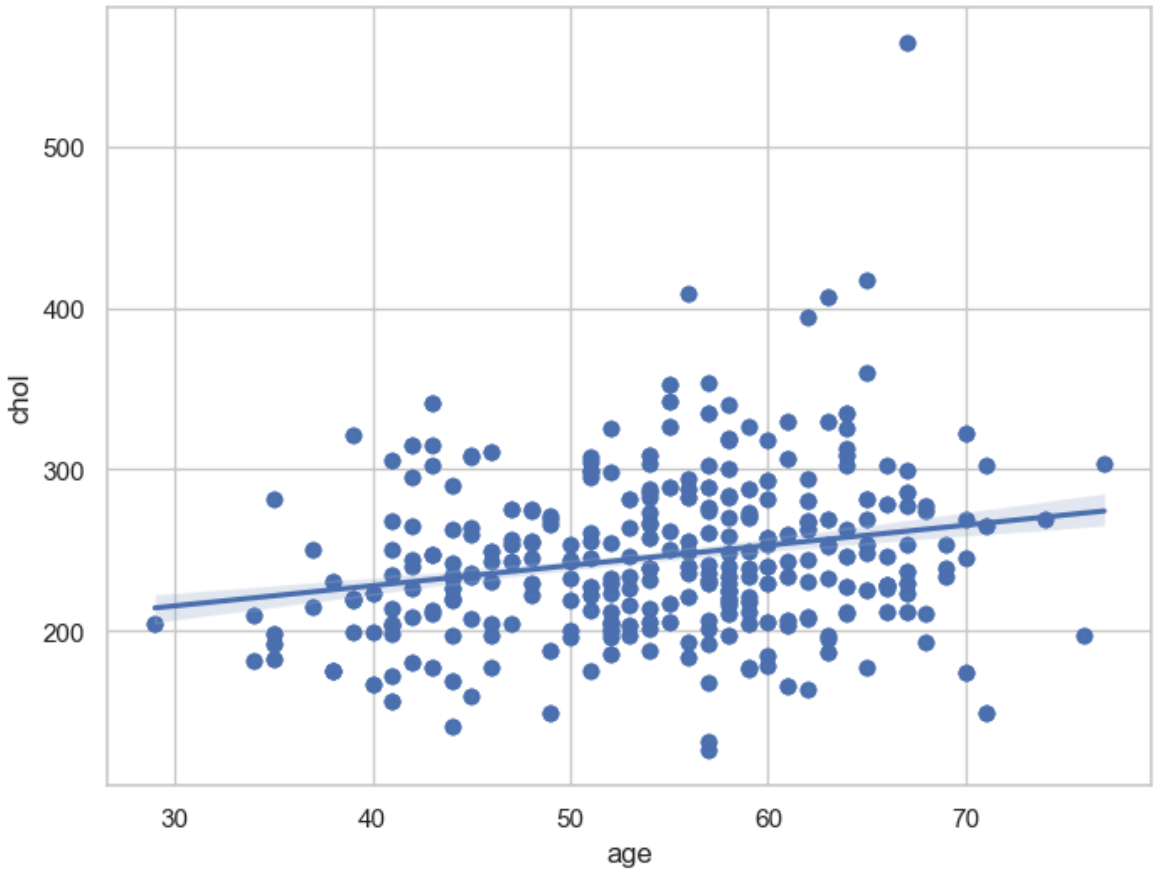
- The above line shows that linear regression model is not good fit to the data.

Analyze `age` and `chol` variable

In [50]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="age", y="chol", data=df)
plt.show()
```



In [51]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="chol", data=df)
plt.show()
```

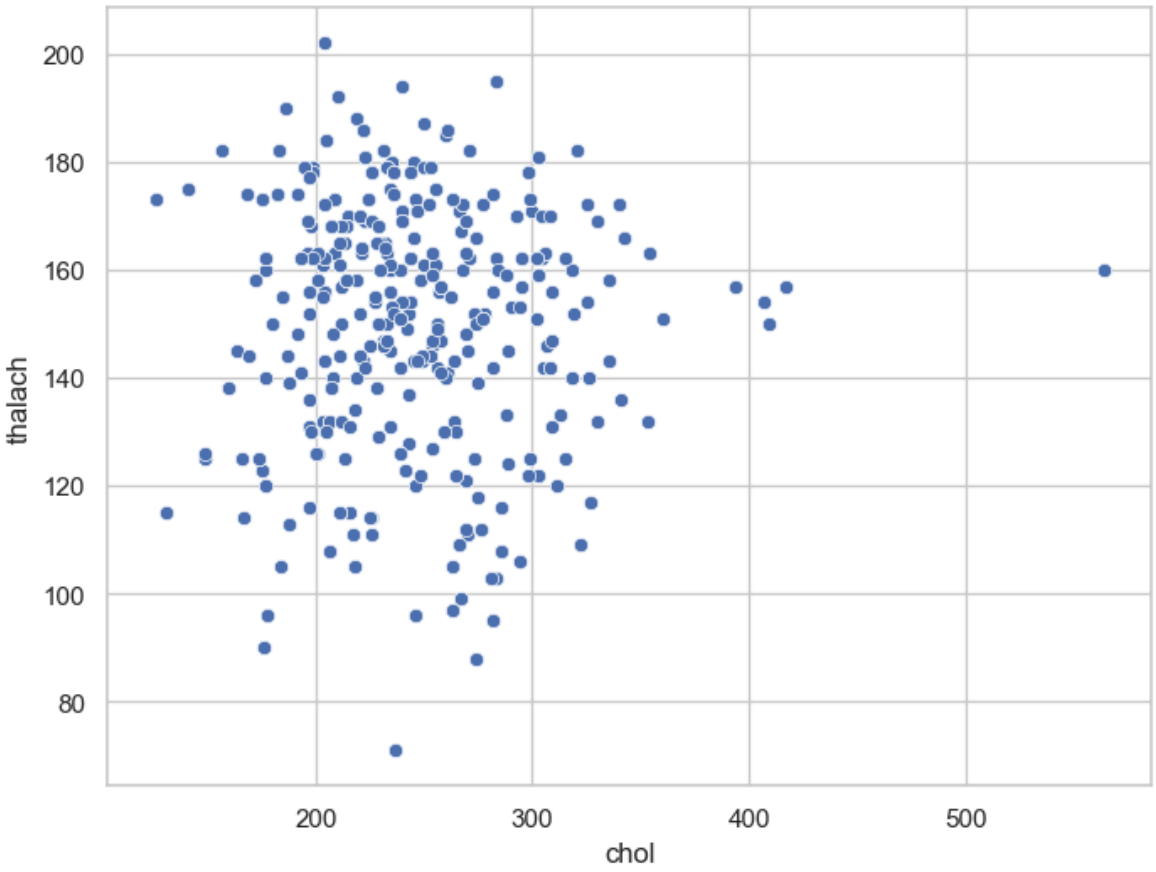


Interpretation

- The above plot confirms that there is a slighly positive correlation between `age` and `chol` variables.

Analyze `chol` and `thalach` variable

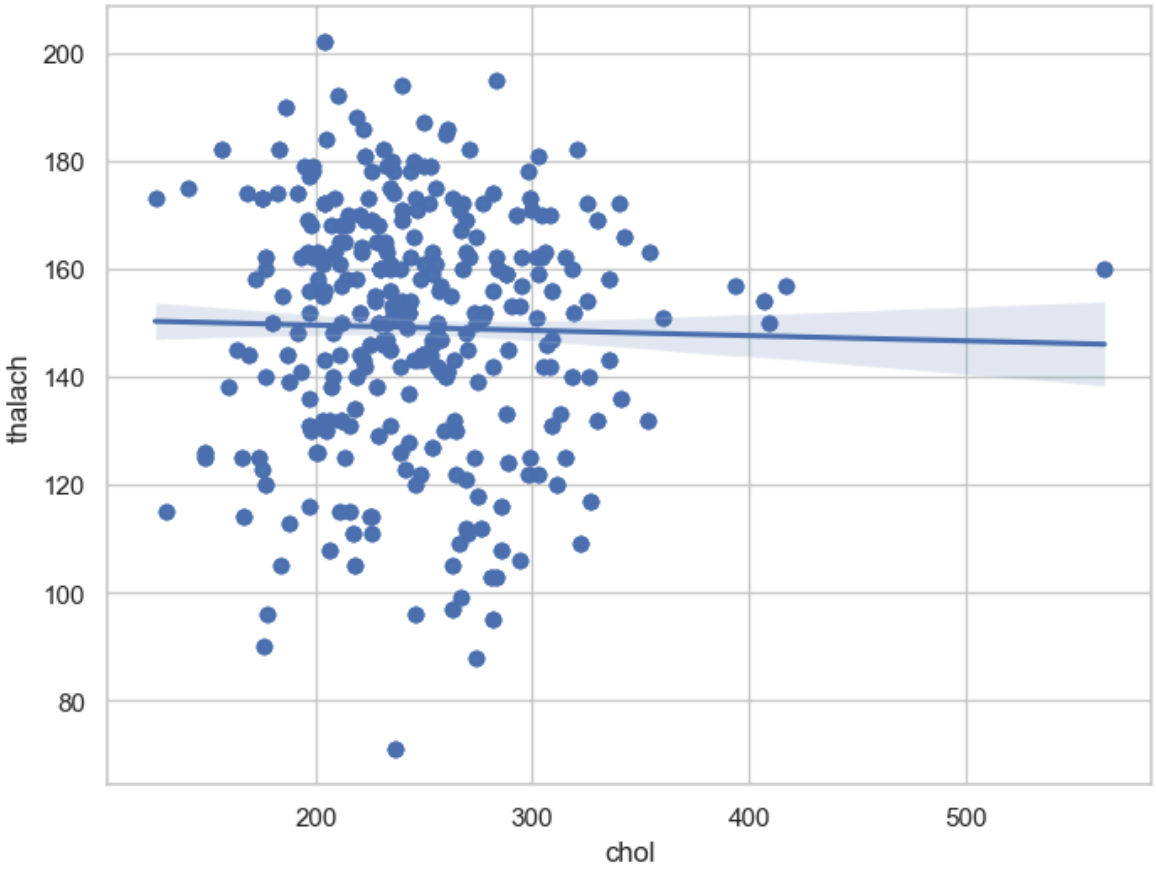
In [52]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="chol", y = "thalach", data=df)
plt.show()
```



Interpretation

- The above plot shows that there is no correlation between `chol` and `thalach` variable.

In [53]:
```
f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="chol", y="thalach", data=df)
plt.show()
```



10. Dealing with missing values

- In Pandas missing data is represented by two values:
  - **None**: None is a Python singleton object that is often used for missing data in Python code.
  - **NaN** : NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation.
- There are different methods in place on how to detect missing values.

Pandas isnull() and notnull() functions

- Pandas offers two functions to test for missing data - `isnull()` and `notnull()`. These are simple functions that return a boolean value indicating whether the passed in argument value is in fact missing data.
- Below, I will list some useful commands to deal with missing values.

Useful commands to detect missing values

- **df.isnull()**

The above command checks whether each cell in a dataframe contains missing values or not. If the cell contains missing value, it returns True otherwise it returns False.

- **df.isnull().sum()**

The above command returns total number of missing values in each column in the dataframe.

- **df.isnull().sum().sum()**

It returns total number of missing values in the dataframe.

- **df.isnull().mean()**

It returns percentage of missing values in each column in the dataframe.

- **df.isnull().any()**

It checks which column has null values and which has not. The columns which has null values returns TRUE and FALSE otherwise.

- **df.isnull().any().any()**

It returns a boolean value indicating whether the dataframe has missing values or not. If dataframe contains missing values it returns TRUE and FALSE otherwise.

- **df.isnull().values.any()**

It checks whether a particular column has missing values or not. If the column contains missing values, then it returns TRUE otherwise FALSE.

- **df.isnull().values.sum()**

It returns the total number of missing values in the dataframe.

In [54]:
```
df.isnull()
```

Out[54]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
0	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...
1020	False	False	False	False	False	False	False	False	False	False	False
1021	False	False	False	False	False	False	False	False	False	False	False
1022	False	False	False	False	False	False	False	False	False	False	False
1023	False	False	False	False	False	False	False	False	False	False	False
1024	False	False	False	False	False	False	False	False	False	False	False

1025 rows × 14 columns

In [56]:
```
df.isnull().any().any()
```

Out[56]:
```
False
```

In [57]:
```
df.isnull().values.any()
```

Out[57]:
```
False
```

In [58]:
```
df.isnull().values.sum()
```

Out[58]:
```
0
```

In [59]:
```
df.isnull().sum()
```

Out[59]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

11. Check with ASSERT statement

- We must confirm that our dataset has no missing values.
- We can write an **assert statement** to verify this.
- We can use an assert statement to programmatically check that no missing, unexpected 0 or negative values are present.
- This gives us confidence that our code is running properly.
- **Assert statement** will return nothing if the value being tested is true and will throw an AssertionError if the value is false.
- **Asserts**
  - assert 1 == 1 (return Nothing if the value is True)
  - assert 1 == 2 (return AssertionError if the value is False)

In [ ]:
```
assert pd.notnull(df).all().all() # assert will not return anything if the condi
```

In [62]:
```
assert(df>=0).all().all()
```

In [63]:
```
assert(df<=0).all().all()
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[63], line 1
----> 1 assert(df<=0).all().all()

AssertionError:
```

Interpretation

- The above two commands do not throw any error. Hence, it is confirmed that there are no missing or negative values in the dataset.
- All the values are greater than or equal to zero.

Outlier Detection

I will make boxplots to visualise outliers in the continuous numerical variables : -

`age` , `trestbps` , `chol` , `thalach` and `oldpeak` variables.

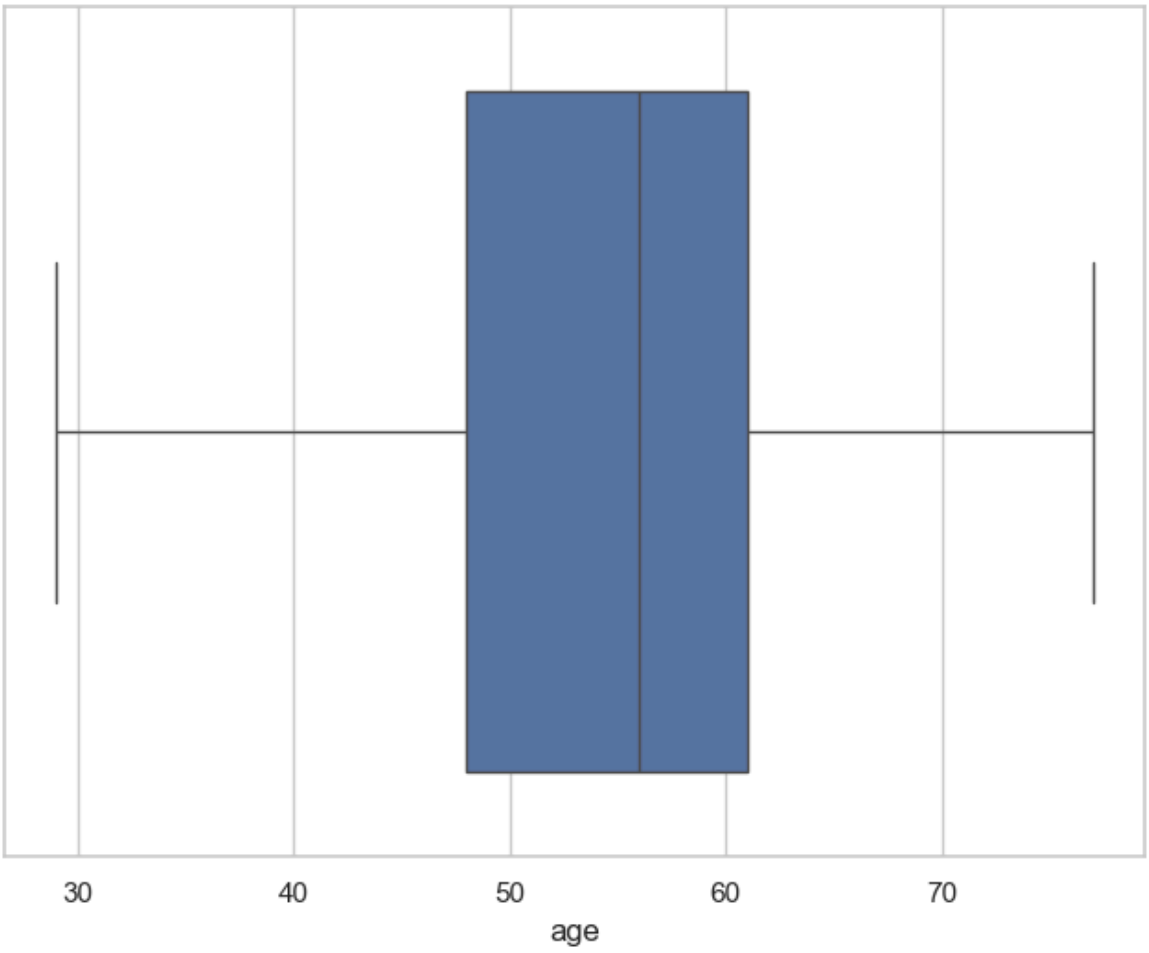
`age` variable

In [64]:
```
df['age'].describe()
```

Out[64]:
```
count    1025.000000
mean       54.434146
std         9.072290
min        29.000000
25%        48.000000
50%        56.000000
75%        61.000000
max        77.000000
Name: age, dtype: float64
```

Box-plot of `age` variable

In [66]:
```
f,ax=plt.subplots(figsize=(8,6))
ax = sns.boxplot(x=df['age'])
plt.show()
```



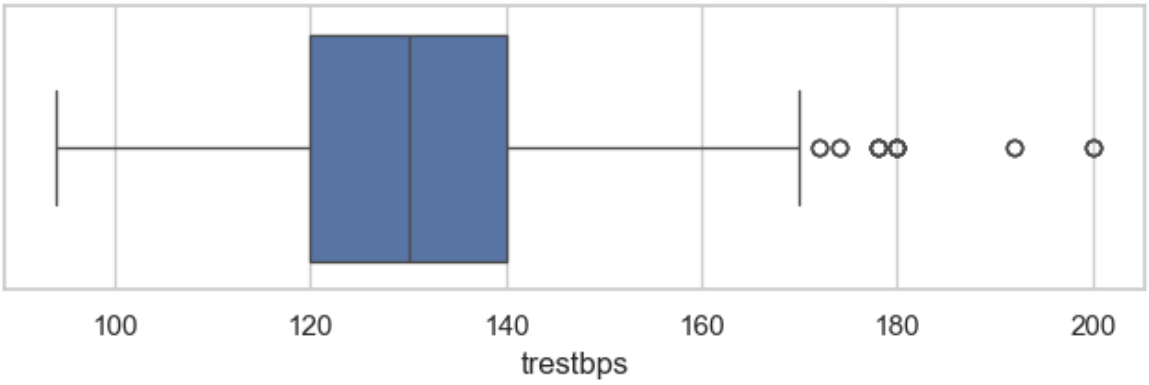
`trestbps` variable

In [67]:
```
df['trestbps'].describe()
```

Out[67]:
```
count    1025.000000
mean      131.611707
std        17.516718
min        94.000000
25%       120.000000
50%       130.000000
75%       140.000000
max       200.000000
Name: trestbps, dtype: float64
```

Box-plot of `trestbps` variable

In [68]:
```
f,ax=plt.subplots(figsize=(8,2))
ax=sns.boxplot(x=df['trestbps'])
plt.show(

)
```



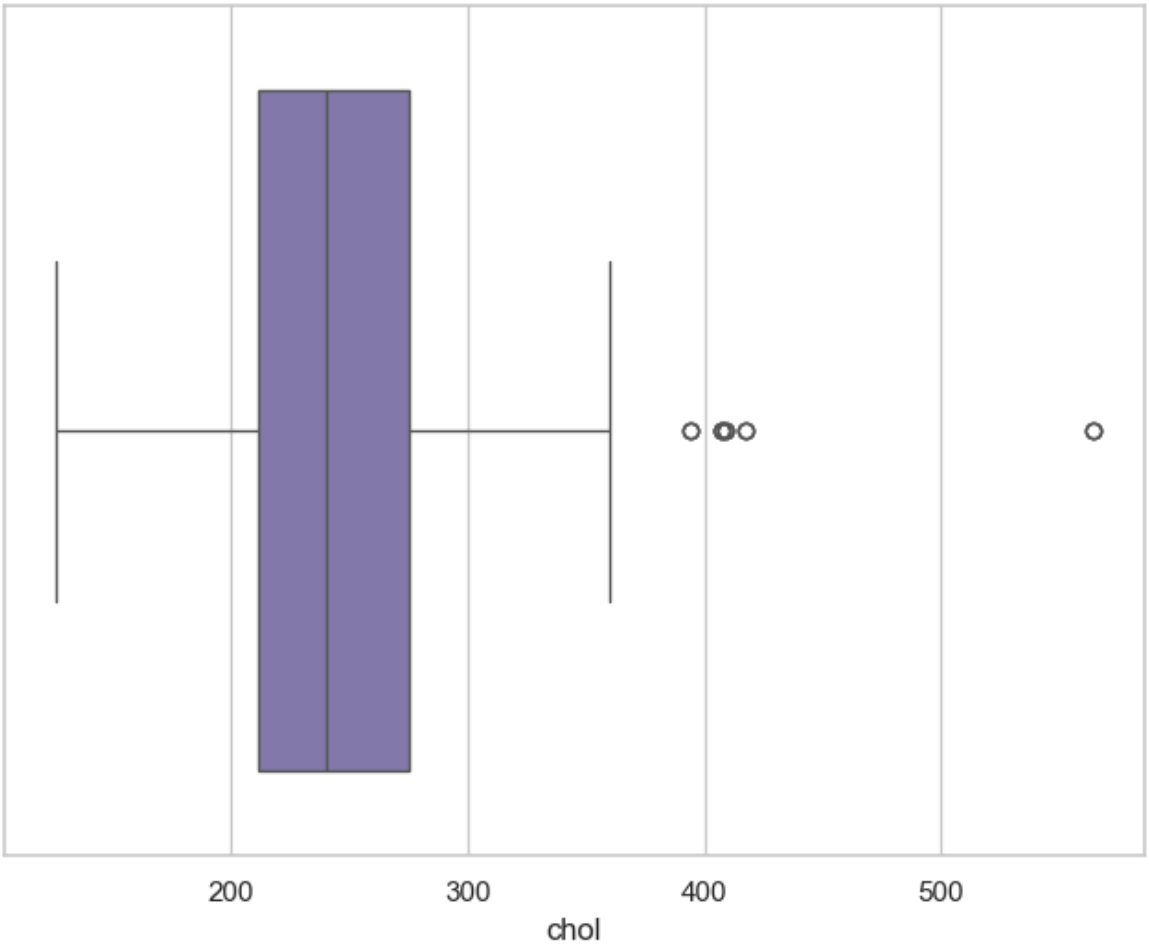
`chol` variable

In [70]:
```
df['chol'].describe(

)
```

Out[70]:
```
count    1025.00000
mean      246.00000
std        51.59251
min       126.00000
25%       211.00000
50%       240.00000
75%       275.00000
max       564.00000
Name: chol, dtype: float64
```

Box-plot of `chol` variable

In [72]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['chol'], color='m')
plt.show(

)
```



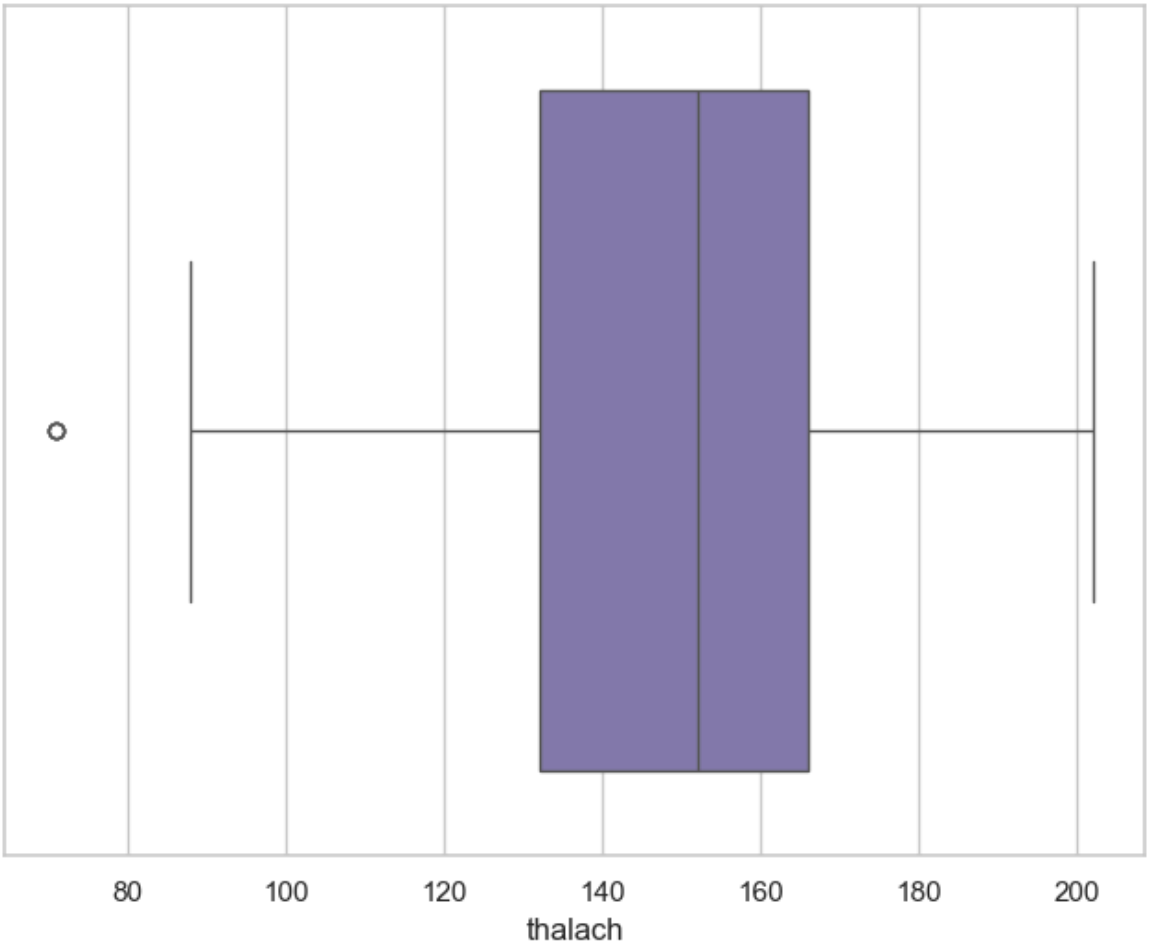
`thalach` variable

In [73]:
```
df['thalach'].describe()
```

Out[73]:
```
count    1025.000000
mean      149.114146
std        23.005724
min        71.000000
25%       132.000000
50%       152.000000
75%       166.000000
max       202.000000
Name: thalach, dtype: float64
```

Box-plot of `thalach` variable

In [76]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['thalach'], color='m')
plt.show(

)
```



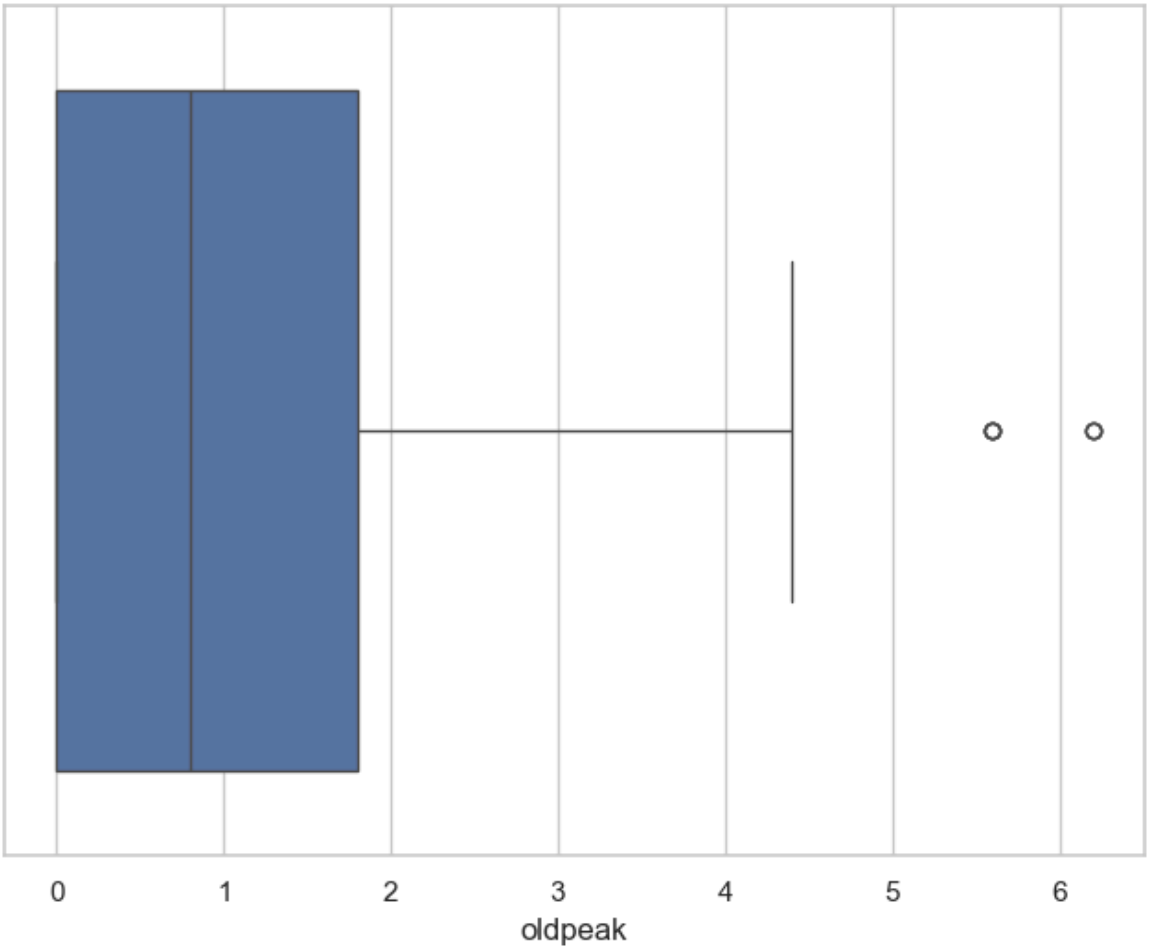
`oldpeak` variable

In [77]:
```
df['oldpeak'].describe()
```

Out[77]:
```
count    1025.000000
mean        1.071512
std         1.175053
min         0.000000
25%         0.000000
50%         0.800000
75%         1.800000
max         6.200000
Name: oldpeak, dtype: float64
```

Box-plot of `oldpeak` variable

In [78]:
```
f,ax=plt.subplots(figsize=(8,6))
ax=sns.boxplot(x=df['oldpeak'])
plt.show(

)
```



Findings

- The `age` variable does not contain any outlier.
- `trestbps` variable contains outliers to the right side.
- `chol` variable also contains outliers to the right side.
- `thalach` variable contains a single outlier to the left side.
- `oldpeak` variable contains outliers to the right side.
- Those variables containing outliers needs further investigation.